

Meerkat: A framework for Dynamic Graph Algorithms on GPUs

Kevin Jude Concessao¹, Unnikrishnan Cheramangalath¹, MJ Ricky Dev², and
Rupesh Nasre²

¹Indian Institute of Technology Palakkad

²Indian Institute of Technology Madras

Abstract

Graph algorithms are challenging to implement due to their varying topology and irregular access patterns. Real-world graphs are dynamic in nature and routinely undergo edge and vertex additions, as well as, deletions. Typical examples of dynamic graphs are social networks, collaboration networks, and road networks. Applying static algorithms repeatedly on dynamic graphs is inefficient. Further, due to the rapid growth of unstructured and semi-structured data, graph algorithms demand efficient parallel processing. Unfortunately, we know only a little about how to efficiently process dynamic graphs on massively parallel architectures such as GPUs. Existing approaches to represent and process dynamic graphs are either not general or inefficient. In this work, we propose a library-based framework for dynamic graph algorithms that proposes a GPU-tailored graph representation and exploits the warp-cooperative execution model. The library, named **Meerkat**, builds upon a recently proposed dynamic graph representation on GPUs. This representation exploits a hashtable-based mechanism to store a vertex’s neighborhood. **Meerkat** also enables fast iteration through a group of vertices, such as the whole set of vertices or the neighbors of a vertex. We find that these two iteration patterns are common, and optimizing them is crucial for achieving performance. **Meerkat** supports dynamic edge additions and edge deletions, along with their batched versions. Based on the efficient iterative patterns encoded in **Meerkat**, we implement dynamic versions of the popular graph algorithms such as breadth-first search, single-source shortest paths, triangle counting, weakly connected components, and PageRank. Compared to state-of-the-art dynamic graph analytics framework HORNET, **Meerkat** is $12.6\times$, $12.94\times$, and $6.1\times$ faster, for query, insert, and delete operations, respectively. Using a variety of real-world graphs, we observe that **Meerkat** significantly improves the efficiency of the underlying dynamic graph algorithm. **Meerkat** performs $1.17\times$ for BFS, $1.32\times$ for SSSP, $1.74\times$ for PAGERANK, and $6.08\times$ for WCC, better than HORNET on an average.

1 Introduction

Real-world graphs undergo structural changes: nodes and edges get deleted, and new nodes and edges are added. Handling dynamic updates poses new challenges compared to a static graph algorithm. Efficient handling of these dynamic changes necessitates (i) how to represent a dynamically

changing graph, (ii) how to update only the relevant part of the graph depending upon the underlying algorithm, and (iii) how to map this update effectively on the underlying hardware. These issues exacerbate on massively parallel hardware such as GPUs due to SIMD-style execution, the need to exploit on-chip cache for optimal performance, and nuances of the synchronization protocols to deal with hundreds of thousands of threads. Effective addressing of these issues demands new graph representations, binding of the theoretical and systemic graph processing, and tuning the implementation in a GPU-centric manner. Former research has invented multiple graph representations in diff-CSR [28], SlabGraph [7], faimGraph [37], Hornet [10] and cuStinger [19] to maintain the changing graph structure. The SlabGraph framework [7] proposes the *SlabHash* [5]-based graph data structure and follows a warp-based execution model.

Dynamic graph algorithms can be categorized as (i) incremental wherein nodes and edges are only added, (ii) decremental wherein nodes and edges are only deleted, and (iii) fully dynamic which involves both the incremental and the decremental updates. A few prior works deal primarily with one of these types. For instance, Lacki [24] proposes a decremental strongly connected components algorithm.

Existing solutions to deal with dynamic graphs are plagued with one of the two issues: they apply to certain types of graphs, or they are inefficient at scale. Thus, the solutions may work well for small-world graphs such as social networks but are expensive on road networks which are characterized by large diameters. Central to solving these issues lie two fundamental questions related to storage and compute: how to represent a dynamic graph and how to enumerate through a set of graph elements (such as vertices). Graph representation is crucial because the optimal representation for static processing quickly goes awry with dynamic updates. Thus, due to dynamic edge addition, memory coalescing on GPUs can be adversely affected, resulting in reduced performance. Similarly, two types of iterators are common in graph processing: through all the current graph vertices, and through the latest neighbors of a vertex (which change across updates). Both these operations are so common that we treat them like primitives, whose performance crucially affects that of the underlying dynamic graph algorithm. Note that unlike in the case of a static graph algorithm which may suffer from load imbalance due to different threads working on vertices having differently-sized neighborhoods, the issue of load imbalance is severe in a dynamic graph algorithm, as the load imbalance itself may vary across structural updates, leading to unpredictable performance results. This makes applying optimizations in a blanket manner difficult for dynamic graphs and demands a more careful custom processing. Such customization allows the techniques to apply to different algorithms as well as to different kinds of updates for the same dynamic graph algorithm.

This paper makes the following contributions:

1. We illustrate mechanisms to represent and manipulate large graphs in GPU memory using a hash-table based data-structure. Our proposed dynamic graph framework **Meerkat**, makes primitive operations efficient, such as iterating through the current neighbors of a node, iterating through the newly added neighbors of a node, etc.
2. Using the efficient primitives in **Meerkat**, we demonstrate dynamic versions of popular graph algorithms on GPUs: breadth-first search (BFS), single source shortest paths (SSSP), triangle counting, PageRank, and weakly connected components (WCC). Apart from the common patterns among these algorithms, we highlight their differences and how to efficiently map those for GPU processing.

This project is supported by National Supercomputing Mission, India

3. We qualitatively and quantitatively analyze the efficiency of our proposed techniques implemented in **Meerkat** using a suite of large real-world graphs and four dynamic algorithms (namely, BFS, SSSP, PageRank, and Triangle Counting) and one incremental-only algorithm (namely, WCC). **Meerkat** eases programming the dynamic algorithms and readily handles both the bulk and the small batch updates to the graph object. We illustrate that the dynamic algorithms built on **Meerkat** significantly outperform their static counterparts.
4. Compared to state-of-the-art dynamic graph analytics framework HORNET, **Meerkat** is $12.6\times$ times faster for query, $12.94\times$ faster for bulk insert, and $6.1\times$ for bulk delete operations. **Meerkat** performs $6.08\times$ for weakly connected components, $1.17\times$ for BFS, $1.32\times$ for SSSP, and $1.74\times$ for PAGERANK, better than HORNET on an average.

2 Motivation

Awad et al. [7] propose a dynamic graph data structure (which we shall refer to as SLABGRAPH) that uses the SLABHASH data structure [6] for maintaining the vertex adjacencies. SLABGRAPH exploits a concurrent hashtable per vertex to store adjacency lists using a form of chaining. The data structure is designed and optimized for warp-based execution on the GPU. SLABGRAPH allocates a SLABHASH object for each vertex. A SLABHASH object has a fixed number of *buckets*, determined a priori by the *load-factor* and the number of adjacent vertices. Each bucket corresponds to a slab list: a linked list of *slabs*. Each slab is 128 bytes long, to closely match the L1 cache line size, for coalesced memory access within a single warp [6]. The adjacent vertices of a source vertex are stored in one of the buckets determined by a hashing function. The 128 bytes in a slab form 32 lanes with 4 bytes per lane (32 is the GPU’s warp size). Each lane is to be processed by a corresponding thread in the warp. The last lane is reserved for storing the address of the next slab. SLABHASH’s CONCURRENTSET (CONCURRENT MAP, respectively) is used for unweighted (weighted, respectively) graphs to store the adjacent neighbours for each vertex. Every slab in the CONCURRENTSET can be used to store up to 31 neighbouring vertices. The last lane in a CONCURRENTSET slab is reserved for storing the address of the next slab. While all threads participate in retrieving a single CONCURRENTSET slab, only 31 threads participate actively in query/traversal operations, since their corresponding slab lanes potentially could have vertex data. The last thread fetches the next slab’s address and is used for performing traversal to the next slab. A CONCURRENTMAP slab used for a weighted graph can store up to 15 pairs of the neighboring vertices and their respective edge-weights adjacent elements, as the graphs are weighted. It can store up to 15 pairs of the neighboring vertices and their respective edge weights. When a slab is retrieved by a warp, every pair of a neighboring vertex and an edge weight is fetched by a pair of threads. While 30 threads are involved in fetching edge-related data, only 15 threads are involved in processing 15 pairs in the CONCURRENTMAP slab. Like the case of a CONCURRENTSET slab, the last thread fetches the next slab’s address and is used for performing a traversal to the next slab. A graph with an average degree greater than 15 would allocate at least $2.1\times$ more slabs for the weighted SLABGRAPH representation (with CONCURRENTMAP), than the unweighted representation (with CONCURRENTSET), requiring at least $2.1\times$ more slab retrievals in processing weighted representation. The weighted representation has 48.4% processing efficiency compared to the unweighted representation in a full graph traversal operation. An **EMPTY_KEY**¹ is stored in a slab lane if it has not been populated with an adjacent

¹EMPTY_KEY is defined as `UINT32_MAX-1` for `ConcurrentSet` and `UINT64_MAX-1` for `ConcurrentMap`

vertex previously, and with a special `TOMBSTONE_KEY`² if the slab lane previously held a valid vertex, and is now deleted. Elements within a slab are *unordered* allowing efficient concurrent access. A slab can be processed efficiently by all the threads of a warp by using warp-wide communication intrinsics such as `__ballot_sync`, `__shfl_sync`, and `__shfl_down_sync` [29]. The warp-cooperative work strategy (WCWS) for searching in a `SLABHASH` hash table is described by [5] and the same is used in the **Meerkat** framework.

The `SLABGRAPH` data structure provides efficient ways for the insertion and deletion of edges on dynamic graph objects. Unlike other dynamic graph data structures, such as `STINGER` [15] and `HORNET` [10], only `SLABGRAPH` relies on Warp Cooperative Work Sharing (WCWS) execution model [6]. The `SLABGRAPH` data structure has the below shortcomings.

Memory Allocation: The original `SLABHASH` data structure assumes the responsibility of allocating the head slabs via `cudaMalloc` in `CONCURRENTMAP` or `CONCURRENTSET`. Thus, when a `SLABHASH` object is associated with each vertex, and when at least one head slab for each vertex, even when the vertex has no incident edges. This is required for maintaining the dynamic graph capabilities of `SLABGRAPH`. Since real-world input graphs often have millions of vertices, we observed that a large number of `cudaMalloc` calls (as many as the number of vertices) for a slab of size 128 bytes results in a significant explosion in the total memory allocated.

Traversal of Edges: Programming dynamic algorithm becomes easy when the underlying framework provides different iterators to traverse through all the edges in the graph. This is missing in `SLABGRAPH`. Developing an edge-centric algorithm (such as Kruskal’s minimum spanning tree or Karger’s Min-Cut based on edge contractions) without such a facility is difficult.

WarpLevel APIs: Understanding and using low-level warp primitives is involved. The WCWS can be made easy with abstractions for primitive operations such as reduction and communication within a warp. This is missing in `SLABGRAPH`.

Auxiliary Data Structures: Programming different dynamic algorithms is made easy with the help of auxiliary data structures that work on top of the dynamic graph data structure. This is missing in `SLABGRAPH`.

These shortcomings in `SLABGRAPH` motivated us to implement **Meerkat**. **Meerkat** makes programming dynamic graph algorithms easy and we were able to program efficient dynamic graph algorithms for SSSP, BFS, PR, and TC. We did a quantitative and qualitative analysis of our results with `Hornet`, a state-of-the-art dynamic graph framework.

3 Meerkat Framework

Our work **Meerkat** builds and improves upon `SLABGRAPH` [7] by extending the publicly available source code for `SlabHash`¹. Figure 1 shows the extensions done by us to `SLABGRAPH` in our framework **Meerkat**.

Dynamic graph algorithms on GPUs demand two crucial considerations: memory efficiency due to dynamic updates, and computation efficiency since the dynamic processing should be faster than rerunning the static algorithm on the modified graph. Based on this goal, **Meerkat** offers a two-pronged approach. In **Meerkat**, we move the responsibility of allocating the head slabs in `SlabHash` outside (to `SLABGRAPH` part of **Meerkat**) for all the vertices, as the framework has a better picture of the overall allocation. Section 6 demonstrates the significant memory savings with this approach.

²`TOMBSTONE_KEY` is defined as `UINT32_MAX-2` for `ConcurrentSet` and a 64-bit pair `(UINT32_MAX-2, UINT32_MAX-2)` for `ConcurrentMap`

¹<https://github.com/owensgroup/SlabHash>

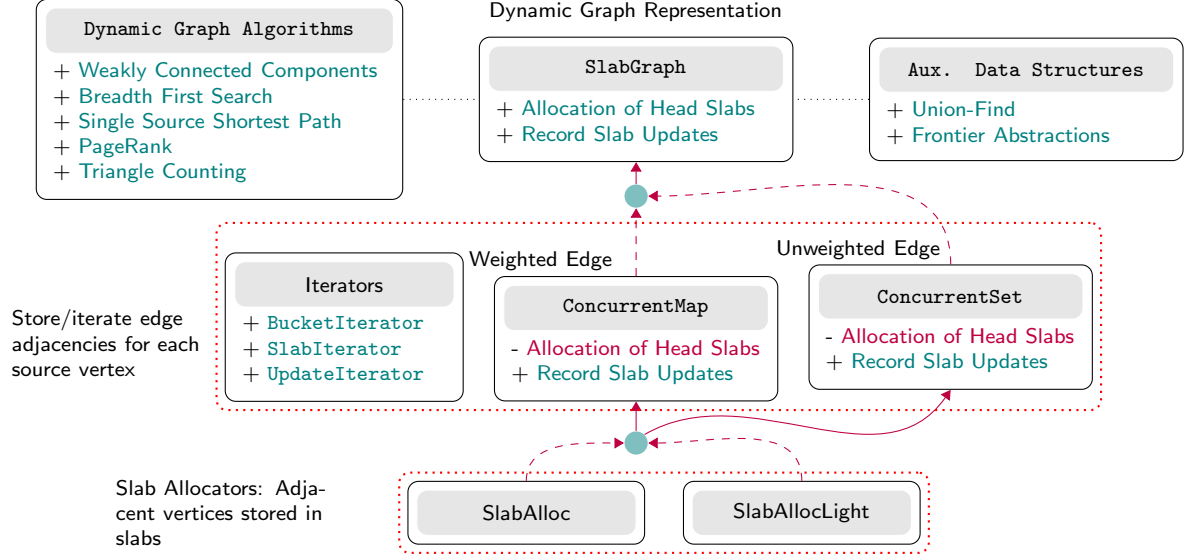


Figure 1: Our proposed **Meerkat** framework and its dependencies. Teal colored text shows our extensions.

Second, **Meerkat** provides a set of iterators for traversing through the neighbors of a vertex, which is a fundamental requirement for almost all graph algorithms, such as BFS and SSSP. **SLABGRAPH** [7] focuses mainly on the representation and operations of dynamic graphs. In many incremental algorithms such as weakly-connected components, it is sufficient to process the updates performed on the graph representation. Our iterators in **Meerkat** (see Section 3.4) enable us to traverse through individual buckets selectively, through all the slab lists for a vertex, or visit only those slabs holding new updates, depending on the requirements of the underlying dynamic graph algorithm.

Third, The **Meerkat** framework provide abstractions for *warp-level* APIs such as *reduction*, and *broadcast*. The **Meerkat** framework comes with auxiliary data structures such as **Frontier**, and **UNION-FIND** to ease programming dynamic graph algorithms.

3.1 Memory Management in Meerkat

Meerkat moves the responsibility of allocating the head slabs from **SLABHASH** to the **SLABGRAPH** object which decides the number of slabs required per vertex according to the load factor (See Figure 1). A single large array of **head_slabs** is allocated using a single **cudaMalloc()** function call. Each vertex is assigned a specific number of head slabs according to the initial degree. We maintain an array (**bucket_count**) such that **bucket_count[v]** is the initial number of head-slabs allocated for a given vertex v . By performing an **exclusive_scan** operation on the entries of the **bucket_count** array, we can determine the offset to the to the head slab for each vertex within the **head_slabs** array. Each **SLABHASH** object for a vertex maintains a unique context object, which stores device pointers to these allocated slab lists, and is, therefore, shallowly copied to the device’s global memory. Every graph search/operation index into an array of **SLABHASH** context objects, to retrieve the object for the source vertex. Using the hash function, the particular slab list which

is to store the destination vertex is determined. This slab list is then linearly traversed by the warp which has the source vertex in its work queue. When a bucket is full, the underlying SLABHASH data structure invokes a custom allocator to obtain a new slab.

Similar to the SLABHASH implementation, a SLABGRAPH object maintains a device context object that is shallowly copied into the device memory. While the vertex adjacencies are represented and accessible through these SLABHASH context objects, the SLABGRAPH context object provides clean API access for vertex adjacency access and graph manipulation operations inside a device kernel by utilizing a warp-cooperative work strategy. The SLABHASH context object supports methods such as `Insert()` and `Delete()` that execute in a warp-cooperative fashion. These methods are internally used by SLABGRAPH's device API's such as `InsertEdge()` and `DeleteEdge()`, for inserting and removing adjacent vertices for a specific vertex, respectively.

Algorithm 1: Meerkat Warp device APIs:
dequeue and reduction

```

1 __device__ int warpdequeue (bool
  *to_process) {
2   int work_queue =
    __ballot_sync(0xFFFFFFFF,
      *to_process)
3   index = __ffs (work_queue) - 1
4   if (lane_id() == index) then
5     *to_process = false
6   end if
7   return index
8 }
9 __device__ int warpbroadcastsum(T
  *val) {
10  int i = 1;
11  while (i < 32) do
12    *val += __shfl_xor_sync
      (0xFFFFFFFF, F, *val, i)
13    i = i * 2
14  end while
15  return val
16 }
```

Algorithm 2: Meerkat Warp device APIs:
broadcast, and frontier enqueue

```

1 __device__ void
  warpenqueuefrontier(Frontier<T> frontier,
    T value, bool to_enqueue) {
2   uint32 bit_set =
    __ballot_sync(0xFFFFFFFF, to_enqueue)
3   uint32 current_base = 0
4   if (lane_id() == 0) then
5     current_base = atomicAdd
      (&frontier.size, __popc(bit_set))
6   end if
7   current_base =
    __shfl_sync(0xFFFFFFFF,
      current_base, 0, 32)
8   if (to_enqueue) then
9     uint32 offset =
      __popc(__brev(bit_set) &
        (0xFFFFFFFF << (32 - lane_id)))
10    frontier[current_base + offset] =
      value
11  end if
12 }
13 __device__ int warpbroadcast (T *val, int
  lane_id) {
14  *val = __shfl_sync (0xFFFFFFFF, *val,
    lane_id)
15 }
```

3.2 Warp Level APIs in Meerkat

The warp cooperative work sharing execution model relies on each warp processing the neighbours of the same vertex, using warp intrinsics. The warp maintains a queue of such vertices which are

elected in turn in First in First Out (FIFO) fashion, using lane-id’s of the threads in a warp. **Meerkat** provides an abstraction for such a queue(FIFO) for each warp. This is implemented using the warp level primitives `__ballot_sync()` and `__ffs()`. The pseudocode for the `warpdequeue()` operation of the warp-private queue is given in Algorithm 1. The explanation of the `warpdequeue()` operation is given in Section 3.4 along with Algorithm 3. These API functions abstract the warp level primitives and eases programming dynamic graph algorithms in **Meerkat**. The **Meerkat** framework also provides APIs for warp level reductions and broadcast (see Algorithms 1 and 2).

The **Meerkat** framework targets GPU, and the *device* API comes with detailed abstractions for programming dynamic graph algorithms. The neighbors of a vertex *src* can be obtained using the call to the API function `GetEdgeHashCtxts()` with *src* as the argument. The **Meerkat** framework provides different types of iterators to traverse over the adjacent vertices for a vertex. These iterators are named as **SlabIterator**, **BucketIterator**, and **UpdateIterator** (See Section 3.4). These iterators come with functions `begin()`, `end()`, `beginAt()`, and `endAt()`. These functions are briefed in Table 1.

API	Parameters	Description
<code>begin()</code>	-	Returns a SlabIterator to the first slab, in the first slab list
<code>end()</code>	-	Returns an invalid SlabIterator to a logically invalid slab (that is, at <code>INVALID_ADDRESS</code>)
<code>beginAt()</code>	<code>index</code>	Returns a BucketIterator to the first slab in the <code>index</code> ’th slab list for the source vertex
<code>endAt()</code>	<code>index</code>	Returns an invalid BucketIterator to a logically invalid slab in the <code>index</code> ’th slab list
<code>updateBegin()</code>	-	Returns an UpdateIterator to the first slab holding incremental updates. The iterator is invalid if updates are not available for the source vertex.
<code>updateEnd()</code>	-	Returns an invalid UpdateIterator to a logically invalid slab (that is, at <code>INVALID_ADDRESS</code>)

Table 1: **Meerkat**: Iterator API of **SlabHashCtxt** representing a source vertex.

3.3 Auxiliary Data Structures

3.3.1 Union-Find

In **Meerkat**, the static and incremental WCC implementation uses the union-find approach for discovering weakly-connected components. **Meerkat** provides the UNION-ASYNC strategy [2] for the UNION operation for the adjacent edges discovered, and full path compression for determining the representative elements for the vertices in FIND operation.

3.3.2 Frontier

A frontier of type $F<T>$ is internally an array of elements of type T . Each frontier object supports integer-based indexing for accessing its elements by our kernel threads. Every frontier object maintains a *size* attribute to indicate the number of elements in the frontier array. Insertion of elements into a frontier object is performed by the warp-cooperative `warpenqueuefrontier()` function. All threads in the warp must be active for its correct invocation. The function `warpenqueuefrontier()` takes a *frontier* object, a *value* to be inserted into *frontier*, and a boolean *to_enqueue* predicate to indicate whether the invoking thread has element participates to insert an element into the

frontier. After the bitset of participating threads is computed with a `_ballot_sync` (line 2), the first warp thread increments the size of the frontier by the number of elements to be inserted by the warp (lines 4-5). The base offset obtained by the first thread (line 5), is broadcasted to all the warp threads (line 7). Each participating thread writes its element into the frontier (line 10) by counting the number of participating threads present in the warp positionally before itself (line 9).

Our implementations for the BFS and SSSP algorithms on **Meerkat**, both static and dynamic, rely on using a pair of frontiers for driving their iterations: a frontier $f_{current}$ holding a set of edges whose destination vertices must be inspected; the outgoing edges from these destination vertices which have been updated populate the frontier f_{next} to be used for the next iteration.

3.4 Graph Primitives

One of the primitive graph operations is to iterate through the neighbors of each vertex. Our **Meerkat** framework maintains three types of iterators: **SlabIterator**, **BucketIterator**, and **Update-Iterator** (see Table 2). **UpdateIterator** is an optimized version of **SlabIterator** customized for incremental-only graph processing (no deletions).

The unit of access for all our iterator variants is a slab. Both the **CONCURRENTSET** slab and the **CONCURRENTMAP** have the same size and store the next slab’s address at identical locations. Consequently, our iterators have been designed to be decoupled from the implementation of our backing stores (**CONCURRENTSET**/**CONCURRENTMAP**): they expose the same API (see iterator-specific methods in Table 3) for both the weighted and unweighted representation of **Meerkat**. Our iterators behave identically in the manner of traversal of slabs and in the retrieval of slab content, regardless of whether **CONCURRENTSET** or **CONCURRENTMAP** is used for storing the neighbors of a vertex.

A **BucketIterator** is constructed for a specific slab list in the slab-hash table. The **SlabIterator** is an abstraction over the **BucketIterator**: the **SlabIterator** internally maintains a **BucketIterator** for the first slab list on construction. When the first slab list has been traversed, it maintains an iterator for the second slab list, and so on, until all the slab lists for a vertex have been fully traversed. The `begin_at(bucket_id)` on a slab hash table constructs an iterator to the first slab of the slab list indexed with `bucket_id`. The `end_at()` method returns an iterator for a logical sentinel slab for the slab list. The slab hash tables expose `begin()` and `end()` methods for retrieving the iterators to the first slab, and to a sentinel slab respectively, for the entire hash table, storing the adjacent vertices.

Both types of iterators are equality-comparable and support the increment operation. The increment operator changes its internal state to refer to the elements in the next slab in sequence. Both the iterators support the `get(lane_id)` method to obtain the element stored at a given `lane_id` of the slab. We use `is_valid_vertex()` to determine if the value returned by `iterator.get(lane_id)` is suitable for processing by our algorithms using these iterators (see Table 3).

We describe two different schemes for enumerating neighbors. Algorithm 3 describes the first iteration scheme, namely *IterationScheme1*. The CUDA kernel accepts a dynamic graph G , and an array of vertices A_v whose adjacencies in the graph G are to be visited. For example, this array A_v could be holding a frontier of vertices in the *BFS* algorithm whose adjacencies have to be visited in a given iteration. The CUDA kernel is invoked with t threads where, $t = \left\lfloor \frac{A_v.length + BS - 1}{BS} \right\rfloor$. **BS** refers to the thread-block size chosen for the kernel invocation and must be equal to a multiple of the warp-size. The warp-size is equal to 32 for the GPU we used for experimental evaluation. In other words, the kernel is invoked with a number of threads equal to the smallest multiple of

Iterator	Description
SlabIterator	traverses through all the slabs contained in the slab lists for a given vertex, one slab list at a time
BucketIterator	our primitive form of SlabHash iterator; traverses through all the slabs of a single slab-list only
UpdateIterator	traverses through only those slabs containing new adjacent vertices, contained in updated slab-lists

Table 2: Meerkat iterators

Function	Description
Meerkat Iterator-specific methods	
<code>iter.operator++()</code>	Advances the iterator to the next slab in sequence.
<code>iter.get_pointer()</code>	accepts a lane-id (0-31), returns a pointer to an element within a slab with an offset of lane-id
<code>iter.first_lane_id()</code>	used when <code>iter</code> is an UpdateIterator ; returns laneid of the first new neighbor in the slab
Meerkat context object-specific methods	
<code>G.get_vertex_adjacencies()</code>	returns pointer to a device vector of SlabHash objects; <code>i</code> 'th element has neighbors of <code>i</code> 'th vertex
<code>begin()</code>	returns a SlabIterator to the first slab, in the first slab list
<code>begin_at(i)</code>	returns a BucketIterator to the first slab in the <code>i</code> 'th slab list
<code>update_begin()</code>	returns an UpdateIterator to the slab at <code>alloc_addr</code> located within the first updated slab list
<code>end(), update_end(), end_at(i)</code>	returns an iterator to a logically invalid slab (that is, at <code>INVALID_ADDRESS</code>)
Meerkat Other intrinsics	
<code>thread_id()</code>	returns global id of calling thread (<code>blockDim.x × blockIdx.x + threadIdx.x</code>)
<code>lane_id()</code>	returns the position of a thread within a warp (in the range [0, 31]) (<code>threadIdx.x & 0x1F</code>)
<code>global_warp_id()</code>	returns the global id for a warp (<code>thread_id() >> 5</code>)
<code>is_valid_vertex()</code>	returns <code>true</code> if <code>v</code> is a valid vertex-id (<code>v ≠ INVALID_KEY</code> and <code>v ≠ TOMBSTONE_KEY</code>)

Table 3: Meerkat primitives

the thread-block size above or equal to $A_v.length$. It is necessary for the thread-block size to be a multiple of the warp size for the successful execution of intra-warp communication primitives such as `_ballot_sync` [29], used for work-cooperative work strategy extensively used in the implementation of our algorithms. The expression $(thread_id() - lane_id())$ finds the thread-id of the first thread in a warp. The predicate at Line 3 allows only those warps which have at least one thread with thread-ids less than the number of vertices in the graph, to proceed with the computation.

The *vertex-id* of the graph object ranges from 0 to $(vertex_n - 1)$. At line 4, we identify those threads whose thread-ids are less than *vertex_n* and can validly index into *V*, the array storing the list of vertices to process.

The `warpdequeue()` function (See Lines 1-7, Algorithm 1) identifies those threads within the warp having a vertex remaining to be processed and stores in the variable *work_queue* (See Line 2, Algorithm 1). Each set bit in the work queue corresponds to one unique thread within the warp that needs to be processed with the value of variable *to_process* == `true`. Using the CUDA function `_ffs()`, we elect the first outstanding thread from *work_queue* and store it in the local variable

Algorithm 3: Iteration Scheme 1 (using SlabIterator)

```
1 function IterationScheme1 (Graph G, Vertex V[vertex_n]) {
2   Vertex_Dictionary* vert_adj[] = G.get_vertex_adjacencies();
3   if ((thread_id() - lane_id) < vertex_n) then
4     bool to_process = (thread_id() < vertex_n);
5     int dequeue_lane = 0; /* queue size is warpsize (i.e 32) */
6     /* dequeue() API internally uses __ballot_sync() and __ffs warp primitive */
7     while ((dequeue_lane = warpdequeue(&to_process)) != -1) do
8       int common_tid = (thread_id() - lane_id) + dequeue_lane;
9       Vertex src = V[common_tid]; /* all warp threads process neighbours of vertex
10                                  src */
11       SlabIterator iter = G.vert_adj[src].begin();
12       SlabIterator last = G.vert_adj[src].end();
13       /* warp cooperative processing of adjacency slabs of vertex src */
14       while (iter != last) do
15         Vertex v = iter.get_pointer(lane_id()); /* each warp thread index to different
16                                                  slab entry */
17         if (is_valid_vertex(v)) then
18           /* Process adjacent vertex, if the slab-entry is not TOMBSTONE_KEY */
19           /*
20           end if
21           ++iter;
22         end while
23         /* Post-processing */
24       end while
25   end if
26 }
```

index (See Line 3, Algorithm 1). The first outstanding bit is the first set bit starting from the least significant bit position. If all the bits in the variable *work_queue* have a value of zero, then the variable *index* will get a value of -1. The local variable *to_process* passed by reference to the *warpdequeue()* function is set to *false* for the warp thread at *lane_id index*. The *warpdequeue()* function then returns the value of the variable *index* (See Line 7, Algorithm 1).

The value returned by the *warpdequeue()* function is assigned to the variable *dequeue_lane* (See Line 6). The *while* loop terminates when the value returned by the *warpdequeue()* function is -1. Thus the loop at Lines 6–17 continues as long as there is an outstanding thread within the warp whose associated vertex is left to process. All the threads within the warp, index into the same position of the *Vertex* array *V*, and the *Vertex* variable *src* will have the same value for all threads in the warp (See Lines 7–8). A pair of *SlabIterators*, namely *iter*, and *last*, are constructed (Lines 9–10) to traverse through the slabs storing the adjacent vertices of the *Vertex src* (within the loop at Lines 11–15). All the threads within the warp perform a coalesced memory access to the contents of the slab represented by *iter* (Line 12). If the value fetched by the thread from the current slab represents a valid vertex, (line 13), the thread processes it as the adjacent vertex. After processing the current slab, the iterator *iter* is incremented so that it refers to the next slab in the sequence.

Unlike *IterationScheme1* which uses *SlabIterators*, *IterationScheme2* (presented in Algorithm 4) uses *BucketIterators* and eliminates the use of a work queue of vertices, and instead

Algorithm 4: Iteration Scheme 2 (using BucketIterator)

```
1 function IterationScheme2 (Graph G, Vertex bucket_vertex[n], int bucket_index[n]) {  
2   VertexDictionary *vert_adjs = &(G.Vert_Dict[0])  
3   int warps_n = (blockDim.x * gridDim.x) / warp_size()  
4   int global_warp_id = thread_id() / warp_size()  
5   int i = global_warp_id  
6   while (i < n) do  
7     Vertex src = bucket_vertex[i]  
8     int index = bucket_index[i]  
9     /* Process source vertex */  
10    BucketIterator iter = vert_adjs[src].begin_at(index)  
11    BucketIterator last = vert_adjs[src].end_at(index)  
12    /* Iterate over adjacent vertices */  
13    while (iter ≠ last) do  
14      Vertex v = *iter.get_pointer(lane_id())  
15      if (is_valid_vertex(v)) then  
16        /* Process adjacent vertex */  
17        end if  
18        ++iter  
19    end while  
20    /* Post-processing */  
21    i += warps_n  
22 end while  
23 }
```

operates with a grid-stride loop. This iteration scheme imposes no restriction on the number of thread blocks. However, for the warp-level primitives (such as `__shfl_sync`) to work correctly on the slabs, the number of active threads within a thread-block must be a multiple of the warp-size (32 on our GPU). Since the adjacencies of a vertex are distributed among multiple slab lists, a slab list can thus be identified with a $\langle v, i \rangle$ pair, which refers to the i^{th} slab-list of a vertex v . Such pairs are stored in the `bucket_vertex` and `bucket_index` device vectors. Each loop iteration within a warp traverses and processes all the slabs contained in one slab-list uniquely identified by its $\langle v, i \rangle$ pair. The $\langle v, i \rangle$ pairs are represented in the `bucket_vertex` and `bucket_index` device vectors. For example, if a vertex frontier contains two vertices v_i and v_j , containing 3 and 2 slab-lists respectively. To enable *IterationScheme2* to traverse through all the their respective slabs, `bucket_vertex[]` is initialized as $[v_i, v_i, v_i, v_j, v_j]$, and `bucket_index[]` is initialized as $[0, 1, 2, 0, 1]$.

The total number of warps in the kernel is computed and stored in `warps_n` (at line 3). Each warp is uniquely identified with a global-warp id (computed at line 4). By using its `global_warp_id` as the initial value for index variable i , each warp identifies its bucket $\langle v, i \rangle$ to process by indexing into the `bucket_vertex` and `bucket_index` vectors (See Lines 7–8). This index is incremented at the stride of the total number of warps in the grid for the CUDA kernel (Line 17).

IterationScheme1 exploits the warp-based processing and schedules one vertex to a warp for processing the neighboring vertices. Although the adjacent vertices within a slab are accessed in a coalesced fashion, the number of slabs inspected is ultimately determined by the degree of the vertex. This leads to an imbalance in the amount of work assigned to each warp if the vertex degree variance is high. This concern is alleviated significantly with *IterationScheme2* since the initial number of buckets for each vertex is determined using the load factor and the initial degree

of that vertex. Further, hashing attempts to distribute the elements uniformly among the buckets. On average, **Meerkat** can distribute the work equally among the warps.

In several incremental graph algorithms, such as incremental WCC, it is sufficient to iterate over slabs for which new adjacent vertices have been inserted. To facilitate iteration over the updated slabs alone, **Meerkat** maintains the following fields per slab list. Each slab list is augmented with a **bool** value **is_updated**, which is set to **true** if new edges are inserted into the slab list. Each slab list stores an allocator address field **alloc_addr** to store the allocator address of the first slab in which new edges have been inserted. Since head slabs are allocated through `cudaMalloc()`, we use a special value **A_INDEX_POINTER** in the **alloc_addr** field, to distinguish the head slab from other slabs returned by the **Meerkat** allocator. Each slab list also stores the lane-id of the first updated value, in the first updated slab.

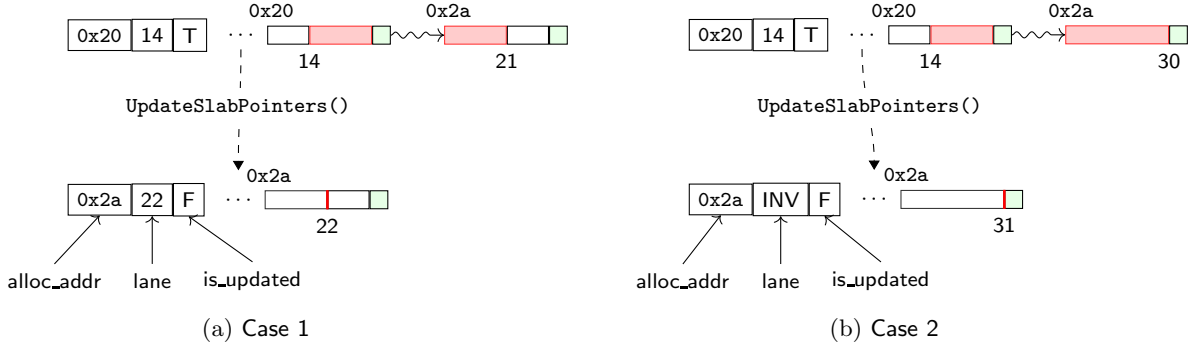


Figure 2: `UpdateSlabPointers()`

Initially, **is_updated** for a slab list is set to **false**. The `InsertEdge()` device method is responsible for setting **is_updated** for a slab list to **true**, if an insertion occurs at the end of the slab list. For every **SLABHASH** object associated with a vertex, we define `UpdateIterators` to iterate over only the slabs storing new vertices. In other words, we can traverse only over those slabs in which new vertices have been inserted. An `UpdateIterator` skips over slab lists for whom **is_updated** is **false**. Once the updates have been processed, `Graph.UpdateSlabPointers()` sets the **is_updated** field to false, for all the slab lists previously set to true. For such slab lists, `Graph.UpdateSlabPointers()` sets **alloc_addr** to the last slab in the slab list, and lane id 1 to the next lane, where subsequent insertions of adjacent vertices are to take place (See Figure 2a). If the slab list is completely full, the lane id field **lane** is assigned a special value **INVALID_LANE** to denote that the updates would occur at newly allocated slabs, chained at the end of the last slab. (See Figure 2b)

Essentially, an `UpdateIterator` behaves like a `SlabIterator`, but, over slabs that are recognized to be holding incremental updates. Hence, like the `SlabIterator`, the use of `UpdateIterators` is only compatible with *IterationScheme1*. In our experiments involving *traversal* of adjacencies of all vertices, *IterationScheme1* (with `SlabIterators`) outperforms *IterationScheme2* (with `BucketIterator`) by 1.24-1.48 $\times$. *IterationScheme2* performs marginally better with algorithms when the working set of vertices is small.

4 Dynamic Algorithms using Meerkat

We evaluate **Meerkat** using the dynamic versions of five fundamental graph algorithms: Breadth First Search (BFS), Single Source Shortest Path (SSSP), Triangle Counting (TC), PageRank (PR), and Weakly Connected Components (WCC). BFS, SSSP, TC, and PR are developed for both incremental and decremental processing, whereas WCC is programmed only for incremental processing. The BFS, PR, TC, and the WCC algorithms operate on unweighted graphs, and **Meerkat** uses **CONCURRENTSET** for storing the adjacencies for every vertex. On the other hand, the SSSP algorithm requires a weighted graph representation, and hence, the **CONCURRENTMAP** is used for representing adjacencies of every vertex, and their respective edge weights. The fully-dynamic versions are implemented with incremental and decremental processing as two computation steps.

4.1 Dynamic PageRank

Algorithm 5: Page Rank - Static / Incremental / Decremental Algorithm

```

1 function ComputePageRank (Graph G, float PageRanks[vertex_n], float
   NewPageRanks[vertex_n], float error_margin = 1e-5, float damping_factor = 0.85, int
   max_iter = 100) {
2   int iterations = 0
3   float NewPageRanks[vertex_n]
4   float ContributionPerVertex[vertex_n]
5   float delta = 1.0
6   float teleport_value = 0.0
7   while (delta > error_margin and iterations < max_iter) do
8     FindContributionPerVertex(PageRanks, G.VertexOutDegrees, ContributionPerVertex)
9     Compute(G, ContributionPerVertex, damping_factor, NewPageRanks)
10    if (find(G.VertexOutDegrees, 0) == true) then
11      FindTeleportProb(G.VertexOutDegrees, &teleport_value)
12      foreach i in 0..(vertex_n - 1) parallel do
13        NewPageRanks[i] += damping_factor × teleport_value
14      end foreach
15    end if
16    FindDelta(PageRanks, NewPageRanks, &delta)
17    PageRanks ← copy(NewPageRanks)
18    ++ iterations
19  end while
20 }
```

The PageRank algorithm assigns a score to every vertex in the range $[0, 1]$, which determines its importance in the input graph object. The PageRank value of a vertex can be understood as a probability that a random walk in the graph (with N vertices), will arrive at that vertex, computed by an iterative application of equation 1 for all vertices in a sequence of super-steps until a steady state/convergence condition is met [4].

$$PR_i[v] = \frac{1-d}{N} + d \cdot \sum_{u \rightarrow v} \frac{PR_{i-1}[u]}{out[u]} \quad (1)$$

The pseudocode for static/dynamic PageRank is discussed in Algorithm 5. Algorithm 5 accepts a dynamic graph object G , and an array $PR[vertex_n]$ which identifies the PageRank value for each vertex in the input graph object. In the case of the static algorithm, each element in the array $PR[vertex_n]$ is initialized with the value $\frac{1}{vertex_n}$. In the incremental/decremental case, the array element $PR[v]$ contains the PageRank value of the vertex v , computed before insertion/deletion. Each iteration of the loop (in lines 7-18) represents a "super-step". The PageRank values of iteration i , are determined from those computed in iteration $i - 1$. The maximum number of iterations is upper bounded by max_iter . The iterations continue until $delta = \sum_{v \in V} |PR_i(v) - PR_{i-1}(v)| > error_margin$. In other words, $delta$ is the L1-NORM between the PageRank vectors $\mathbf{PR}_i$ and $\mathbf{PR}_{i-1}$, and is computed at line 16. Ordinarily, computing the ratio $\frac{PR_{i-1}[u]}{out[u]}$ requires two divergent memory access per every incoming $u \rightarrow v$, for computing the PageRank $PR_i[v]$ for vertex v . This ratio does not change in a "super-step". By caching these ratios for every vertex, we can reduce the divergent memory accesses down to one, during the PageRank computation. The *FindContributionPerVertex* GPU kernel in line 8, initializes $Contribution_i[u] = \frac{PR_{i-1}[u]}{out[u]}$ for each vertex u , which can be performed with coalesced memory access.

The new PageRank values are computed in line 9 according to equation 1 and are adjusted to account for teleportation from zero-outdegree vertices to any other vertex in the input graph object, at lines 10-13, on the GPU. The teleportation probability is added to the PageRank value for every vertex (lines 12-13), if there exists any vertex v_z whose out-degree is zero (line 10). The teleportation probability to be computed at iteration i is given by $\sum_{v_z} \frac{PR_{i-1}[v_z]}{vertex_n}$ is computed in the *FindTeleportProb* GPU kernel.

The *Compute* GPU kernel is invoked with a dynamic graph object G storing incoming edges, an array of PageRank contributions for each vertex, the damping factor, and an array of new PageRank values. Each thread, with thread-id equal to v , represents a unique vertex v in the graph object G . Hence, each thread maintains a local variable to hold the new PageRank value for the vertex v it represents. A pair of *SlabIterator*'s are constructed to traverse the slabs holding the in-edges of vertex v . The accumulation of the contribution of the in-edges to the PageRank of vertex v is commutative. Hence, the *Compute* kernel maintains a thread-local variable *local_prsum* to accumulate the PageRank contributions of the neighboring vertices along the incoming edges encountered by the warp threads for vertex v .

4.2 Dynamic Single Source Shortest Path and Breadth First Search

The single-source shortest path (SSSP) algorithm, described in Algorithm 6, takes a dynamic graph object G , and single source vertex SRC , and computes the shortest path to all other vertices from SRC . In the dynamic setting, Algorithm 6 is batch-dynamic in nature: it takes a sequence of edge *batches*, where each batch is either an incremental or decremental batch. The graph object G undergoes modifications through the application of an insertion/deletion edge batch; the incremental/decremental SSSP algorithm re-computes the shortest paths/distances for the affected vertices in the graph, from the vertex SRC . For each node v , let $P_v = (SRC \rightsquigarrow \dots \rightsquigarrow parent(v) \rightarrow v)$ be the shortest path from the source vertex SRC . Our SSSP algorithm is responsible for computing $< distance_v, parent(v) >$ pair³, where $distance_v$ is the length of the shortest path P_v , and $parent(v)$

³In our implementation, we have represented the pair as a 64-bit unsigned integer, with 32-bits reserved for each half of the pair. This allows us to consistently update both halves of the pair with single 64-bit atomic operations on the CUDA GPUS

Algorithm 6: SSSP - Incremental / Decremental

```
input: Graph  $G$ , Vertex  $SRC$ , EdgeBatches  $batches[batch\_n]$ 
/* Static SSSP */
1 Frontier<Edge>  $F_{current}, F_{next}$ 
2 tree_node  $nodes[vertex\_n]$ 
3 InitializeDistance( $SRC, nodes$ )
4 CreateFrontier( $G, F_{current}, SRC, G.get\_bucket\_count()[SRC]$ )
5 while ( $F_{current}.size \neq 0$ ) do
6   SSSP_Kernel( $G, nodes, F_{current}, F_{next}$ )
7   swap ( $F_{current}, F_{next}$ )
8   zero  $F_{next}$ 
9 end while
/* Dynamic SSSP */
10 zero  $F_{current}, F_{next}$ 
11 for  $b$  in  $batches$  do
12   if ( $b.is\_insertion()$ ) then
13     Incremental Algorithm Prologue:
14      $F_{current} \leftarrow b$ 
15   end if
16   else
17     Decremental Algorithm Prologue:
18     Invalidate( $b.edges, nodes$ )
19     PropagateInvalidation( $nodes, SRC$ )
20     CreateDecrementalFrontier( $G, nodes, F_{current}$ )
21   end if
22   Common Epilogue:
23   while ( $F_{current}.size \neq 0$ ) do
24     SSSP_Kernel( $G, nodes, F_{current}, F_{next}$ )
25     swap ( $F_{current}, F_{next}$ )
26     zero  $F_{next}$ 
27   end while
28 end for
```

is the predecessor to the vertex v in path P_v . Every vertex v must have a unique $parent(v)$ in its shortest path P_v , which implicitly implies that every vertex v has a unique shortest-path to the source SRC . It is therefore understood, that by identifying the $parent(v)$ for every vertex v , in its shortest path P_v , we are implicitly maintaining a directed tree T_G which is rooted at SRC , such that each edge $e = (u, v) \in T_G$, v is the $parent$ of u in P_v . Our batch-dynamic incremental/decremental algorithm is responsible for maintaining this dependence tree. In the ensuing discussion, a subtree in T_G , rooted at vertex v , will be represented by T_v . A formal discussion on value dependence in shortest distance computation and its representation as a *dependence tree* can be found in [35].

Incremental SSSP

The addition of a new edge (u, v) could only result in $distance_{new}(v) < distance_{old}(v)$, if $distance(u) < distance(parent(v))$. In such a case, the sub-tree T_v is transplanted under a new parent u in T_G . All such shortest paths $P_x = (SRC \rightsquigarrow parent_{old}(v) \rightarrow v \rightsquigarrow x)$, are now $P_x = (SRC \rightsquigarrow u \rightarrow v \rightsquigarrow x)$. Therefore, it is necessary to re-compute the shortest-path distances for all the vertices in the sub-

tree T_v . Our incremental SSSP algorithm takes an incremental batch of edges as the initial frontier for our static SSSP algorithm.

Decremental SSSP

The deletion of an edge (u, v) from the graph G invalidates $distance(v)$ from the source vertex **SRC**, and the shortest paths P_x for all vertices x in the subtree T_v , if the edge (u, v) in G is also an edge in T_G . If $distance(v)$ is invalidated on the deletion of an edge (u, v) , vertex u ceases to be $parent(v)$. This prompts a propagation of invalidations for the shortest distances (from **SRC**) and the parent vertices determined for all vertices in T_v . In effect, the previously computed T_v ceases to exist in T_G . At this juncture, there are three types of vertices in the graph: (i) a set of vertices V_{valid} whose shortest distances and $parent$ information have not been invalidated (ii) a set of vertices $V_{invalid}$ whose shortest distances and $parent$ information have suffered invalidation, as a direct consequence of being destination vertices of deleted edges present in T_G , or indirectly, as a consequence of the propagation of invalidation, and (iii) a set of vertices $V_{unreachable}$ which were not part of T_G owing to an absence of a path from the vertex **SRC** in G . Such vertices in $V_{unreachable}$ will continue to remain unreachable even after a batch of edge deletions. Thus, the shortest paths for vertices in the $V_{invalid}$, still reachable from **SRC**, can be computed by taking all edges (u, v) such that $u \in V_{valid}$ and $v \in V_{invalid}$, as the initial frontier for our static SSSP algorithm.

The specific details of the implementation of the static SSSP, and the incremental/decremental algorithms, presented in Algorithm 6 are explained below:

The static SSSP computation kernel does frontier-based computation: it accepts a frontier of edges $F_{current}$ and until it produces a new frontier F_{next} for the next invocation. The SSSP kernel is repeatedly invoked it produces an empty frontier F_{next} . A frontier of type $F < T >$ is internally an array of elements of type T . Each frontier object supports integer-based indexing for accessing its elements by our kernel threads. Every frontier object maintains a *size* attribute to indicate the number of elements in the frontier array. Insertion of elements into a frontier object is performed by the warp-cooperative `warpenqueuefrontier(.)` function (see Algorithm 2). All threads in the warp must be active for its correct invocation. Line 3 initializes the tree nodes for all vertices: it sets the shortest path distance for all vertices to **INF** (infinity) (0 for the source vertex **SRC**), and their $parent(v)$'s to **INVALID** vertex (**SRC**, for the source vertex **SRC**). Line 4 initializes the initial frontier $F_{current}$ with the outgoing edges of the source vertex **src**.

Lines 5-7 contains the iterative application of the SSSP kernel on the current edge frontier $F_{current}$, to produce the next edge frontier F_{next} . Subsequently, $F_{current}$ is initialized with the newly produced frontier F_{next} (line 7). The iterative process continues until the current frontier for an iteration is non-empty, that is $F_{current}.size \neq 0$ (line 5). Line 6 invokes the SSSP kernel on a frontier of edges $F_{current}$. Each GPU thread t_i of the SSSP kernel is assigned one frontier edge $e = (e.src, e.dst) = (u_f, v_f)$, where $i < F_{current}.size$. The SSSP kernel updates the tree node of vertex v_f to store the distance $d_{new} = distance(u_f) + e_f.weight$ of v_f , along the edge e_f , and the parent vertex u_f . The vertex v_f 's `tree_node` is *atomically* updated to that of d_{src} , if the old shortest path distance of v_f , d_{old} , is greater than d_{new} , or if $d_{old} = d_{new}$ and if $parent(v_f) < u_f$. Using a pair of `SlabIterators`'s for the adjacencies of v_f valid outgoing edges are identified and enqueued into the next edge frontier.

Lines 12-14 defines the prologue for the incremental SSSP algorithm and the Lines 16-20 defines the prologue for the decremental SSSP algorithm. The Lines 23-25 defines the common epilogue for both the incremental and decremental prologues. The epilogue accepts the frontiers produced by

incremental/decremental prologues for each incremental/decremental batch and iteratively applies the static SSSP computation kernel until convergence.

The incremental/decremental BFS algorithm uses the same kernels as that of incremental / decremental SSSP algorithms described in Algorithm 6 (lines 11-27). However, the static algorithm uses a fast *level-based* approach.

4.3 Dynamic Triangle Counting

Our library’s dynamic triangle counting algorithm is adapted from [27], which is based on an inclusion-exclusion formulation. The algorithm consumes a pair of undirected graphs, namely, G_1 and G_2 , and a sequence of *edges*. For each such edge $(u, v) \in \text{edges}$, the the cardinality of the intersection of the *adjacency*(u) in G_1 and *adjacency*(v) in G_2 is computed in a warp-cooperative fashion. Each thread is assigned an edge; the edges assigned to a warp of threads are processed using the warp cooperative work strategy. . After electing the thread whose edge needs processing using the `warpdequeue` function of `Meerkat` the end-points are broadcasted to the warp threads using the `warpbroadcast()` function of `Meerkat`. A pair of `SlabIterators` are constructed to iterate over the neighbours of vertex v in G_2 . For each such adjacent vertex *adj_v* we check if the edge $u \rightarrow \text{adj_}v$ exists. Such an edge indicates the presence of the triangle comprising of vertices $\langle u, \text{adj_}v, v \rangle$, and the thread-local triangle count is incremented by one. It must be remembered that each thread in the warp sees a different *adj_v*; hence detects different triangles, at the same time. The thread-local triangle counts are finally accumulated at warp level using `warpreduxsum()` API of `Meerkat` and then updated to the global variable that stores the total number of triangles.

4.4 Incremental WCC

A Weakly Connected Component (WCC) of an directed graph is a subgraph where all the vertices in the subgraph are reachable from all others vertices in the subgraph. An efficient way to compute the set of all WCCs in a graph object is by using the `Union-Find` data structure [2]. A root-based union-find tree, followed by full path compression can be used efficiently for computing the labels for the vertices, which are representatives of their WCCs, in both the static and the incremental computation.

5 Related Work

In the recent past, multiple frameworks have addressed the challenges dealing with dynamic graphs on GPUs: Hornet [10], CuSTINGER [19], GPMA [32], LPMA [40], aimGraph [38] and faimGraph [37]. The cuSTINGER data structure uses structure-of-arrays (SoA) representation for maintaining edges and large over-provisioned arrays for maintaining the vertex adjacency lists. Hornet [10] maintains several block arrays. Each block has a size of some power of two. A vertex maintains its adjacency list within one such fitting block. On insertion, if the allocated block cannot accommodate the new edges, the adjacency list is migrated to a larger block in another block array. It maintains a vectorized bit-tree for determining if a block array has empty blocks, which is used in reclamation. Further, to identify blocks of a particular block size, Hornet maintains an array of B+ trees.

Related Work	D	F	CPU	GPU
[13], [30], [17], [12], [38]	✓	✗	✓	✗
SlabGraph [6, 7], [32], [40], [37]	✓	✗	✗	✓
[23], [22],[35], [42], [3], [11]	✓	✗	✓	✗
[41]	✗	✓	✗	✓
[15]	✓	✓	✓	
[19], HorNet [10]	✓	✓		✓

D: Data Structure F: Framework 1: CPU 2: GPU

Table 4: Comparison of Related Work

The Packed Memory Array (PMA) [9] data structure that stores elements in a sorted order leaving gaps within the array for accommodating future updates. PMA is maintained as a self-balancing binary tree [16].

The PMA data structure is extended for GPU [32]. This data structure suffers from uncoalesced memory accesses, overheads in obtaining locks, and lower parallelism if threads conflict on the same segment. GPMA+ [32] overcomes these limitations by first sorting the batch updates by their keys and grouping the updates that fall into the same leaf segment to be processed from the leaf to the root. LPMA [40] overcomes the array expansion problem of GPMA+ by using a leveled array for maintaining the dynamic graph updates.

The aimGraph [38] data structure mainly focuses on memory management for handling updates for a dynamic graph. By allocating a single large block of global memory, aimGraph eliminates round trips between the CPU and the GPU for memory allocations. Like aimGraph, faimGraph [37] utilizes a memory manager to handle allocation requests entirely on the GPU.

SlabHash [5, 7] is a GPU-based concurrent dynamic hash table that utilizes a warp-cooperative work strategy to avoid warp divergence. SlabHash supports concurrent update and search queries in a lock-free manner. Like aimGraph, SlabHash also uses a GPU-based dynamic memory allocator (named SLABALLOC) to allocate and deallocate slabs.

The data structures and frameworks targeting CPU are summarized in the first and third row of Table 4 respectively. GraSU [36] extends existing static FPGA-based graph accelerators, with dynamic graph data processing by proposing a programming template (or API) for graph update and computation. The PMA data structure is used for handling dynamic graph updates. GraSU quantifies every vertex depending on its degree and frequency of updates and decides whether it must be cached on on-chip memory.

ConnectIt [14] implements incremental WCC for multi-core CPUs and GConn [20] extends ConnectIt for GPUs. Multiple works have addressed challenges in incremental WCC on GPUs [33, 21, 34]. Dynamic SSSP, BFS, and MST algorithms are programmed using diff-CSR data structure [28]. The computational complexity of dynamic graph algorithms is explored in [31].

HORNET: HORNET maintains several block arrays: Each block array is a sequence of blocks whose size is exactly some power of two. A vertex is assigned one such block which can accommodate all its adjacent vertices, that is, the size of the assigned block is the smallest power of two above the adjacency size. There are multiple block arrays: each array holding blocks of the same size. indicates whether free blocks are available in the block array or not. A new block array for a given block size may be allocated if no free blocks are available. For each new edge for a vertex, a GPU thread searches for free space within the vertex’s edge block. This is warp-divergent in nature. However, if there are additional edges that cannot be added to the block, the new edges are separately queued, a new edge block of a larger size is allocated, the old adjacencies and the

additional edges are added to the new block, and the old block is reclaimed. Similarly, on the deletion of an edge, if the number of valid edges in a block becomes smaller than half its capacity, these edges are migrated to a smaller block, and the original block is reclaimed. In HorNet, the edges to be processed are divided equally among thread blocks and equally among the threads within the thread block. Since all threads within a warp have the same number of edges to process, warp divergence is avoided. However coalesced memory access is not obtained in HorNet.

6 Experimental Evaluation

Graph	#Nodes	#Edges	Average Degree	Maximum Degree	Maximum Diameter	Memory Allocation	
						in SlabHash	in Meerkat
LJournal [8]	4.85M	69M	14	20293	16	2.56	0.97
Rand10M	10M	80M	8	27	11	4.92	1.34
BerkStan [26]	685K	7.6M	11	249	573	0.48	0.25
Wiki-talk [25]	2.4M	5M	2	100022	9	1.32	0.46
Wikipedia	3.4M	93.4M	27	5333	262	2.02	1
Orkut [39]	3.1M	234.4M	76	33313	9	2.37	1.69
USAFull [1]	23.9M	58.3M	2	9	6261	OOM	6.1

Table 5: Input Graphs and Memory Requirement (in GiB) in Meerkat

The experimental evaluation was performed on an NVidia RTX 2080 Ti GPU. The GPU is equipped with 11GB of global memory with a memory bandwidth of 616GB/s, and 4352 CUDA Cores (68 SMs and 64 cores/SM). All the implementations were compiled with `-O3` and `--use_fast_math` flags on the `nvcc` version 11.7 compiler. Table 5 shows the seven publicly available graphs used for our comparison of the static and dynamic graph algorithms, their average/maximum vertex degrees, and their maximum diameters.

Table 5 also compares the memory space requirements (in GiB) when the memory allocation for the graph is performed inside SLABHASH objects versus inside the dynamic graph in Meerkat. We observe $1.4 - 3.67\times$ ($2.33\times$ on average) of memory savings by using the latter strategy. Should the memory allocation be handled inside SLABHASH, the number of `cudaMalloc()` will be equal to the number of slab-lists, which is at least the total number of vertices in the input graph object, resulting in significantly large memory consumption. Meerkat moves the responsibility of allocating the head slabs from SLABHASH to the SLABGRAPH object which decides the number of slabs required per vertex according to the load factor. A single large array of head slabs is allocated using a single `cudaMalloc()` function call. This results in better memory utilization.

We evaluate our implementation for *four* dynamic graph algorithms: Breadth First Search (BFS), Single Source Shortest Path (SSSP), Triangle Counting (TC), and PageRank (PR). We have compared the performance of static versions of the above-mentioned algorithms on Meerkat, against HORNET, a dynamic graph data structure for GPU. The HORNET data structure comes with the implementation of static graph algorithms on full graphs and does not have implementations for *incremental* and *decremental* dynamic graph algorithms. We were unable to compare cuSTINGER [19] with our work, since the publicly available source code does not compile for our GPU’s architecture (with CUDA compute capability 7.5). For the comparison of the performance

³<https://github.com/cuStinger/cuStinger>

benefit of our dynamic algorithms over their static counterparts, we computed the *self-relative-speedup* over **Meerkat**. We define the following quantity: $s_b^n = \frac{static_b^n}{dynamic_b^n}$. $static_b^n$ is the cumulative execution times of the static algorithm applied for a sequence of n incremental/decremental edge updates of size b , measured after each batch. $dynamic_b^n$ is the cumulative execution times of the incremental/decremental algorithm applied for the same sequence of n incremental/decremental edge updates of size b , measured after each batch. We define s_b^n , the speedup of the incremental/decremental algorithm over the static algorithm, as the ratio of $static_b^n$ to $dynamic_b^n$. We have reported speedups s_{10k}^{10} for our dynamic BFS, SSSP, PAGERANK, and TRIANGLE COUNTING algorithms in Figures 7, 8b, and 11b respectively.

The warp-cooperative work sharing execution strategy (WCWS) adopted by **Meerkat** in insertion/deletion/query operations, and our algorithms requires that warp divergence must be avoided. Data is exchanged among threads using warp cooperative functions such as `ballot_sync`, `ffs`, etc. which are fast, as they work only with registers. In WCWS, multiple threads within a warp thread have different tasks (vertices/edges) assigned to them. The warp threads form a queue for processing these tasks (using `ballot_sync`); a task to be processed is collectively elected by the warp (using `ffs`). Each slab occupies 128 bytes, which closely matches the GPU’s L2 cache line size. All warp threads perform coalesced vectorized memory accesses on a slab storing a vertex’s adjacent neighbours. In HORNET the load balance is achieved sequentially distributing edges equally among all the threads spawned for the kernel grid. However, this method affects the coalesced memory access resulting in poorer performance.

As described in Section 5, HORNET migrates the adjacent neighbours of a vertex to a larger edge-block if the current block cannot accommodate incoming edges. In the case of deletion, if the number of adjacent edges is smaller than a threshold, the edges are migrated to a smaller block. This migration of adjacent neighbours does not happen in **Meerkat**. For every new edge $\langle u, v \rangle$, **Meerkat** applies a hashing function on the destination vertex v , to choose which slab-list to store v . New edges are recorded at the end of the chosen slab list, only if the edge was not previously added. This requires a traversal till the end of the slab list to check for previously added identical edges. Hashing distributes the destination vertices among multiple slab lists, implicitly reducing the number of slabs retrieved for checking duplication. If the slab list cannot accommodate a new edge, **Meerkat** obtains a new slab from the pool of pre-allocated slabs, by invoking the slab allocator. The new slab is linked to the end of the slab list, and the new edge is recorded in it.

Figure 3 compares the insertion throughput of loading an entire graph, and for small-batch insertions (2K, 4K, 8K). **Meerkat** performs For small batch insertions of 2K, 4K, 8K, **Meerkat** performs $3.7\times$ - $11.04\times$ better than HORNET. Figure 5 compares the performance of a query benchmark for batches of randomly generated edges, of sizes ranging from 2^{16} to 2^{20} . On average, **Meerkat** performs 2.16 - $6.37\times$ better than HORNET. Figure 4 compares the throughput of the deletion operation for the whole graph and, for small batches (2K, 4K, 8K). For small batch deletions, **Meerkat** is found to perform $10.48\times$ - $16.15\times$ better than HORNET. The performance improvement is due to better coalesced access, and lack of memory block migration in **Meerkat**. The deletion benchmark shows better performance, as the deletion operation only flips a valid entry to `TOMBSTONE_KEY`. Unlike the insertion operation, the adjacent neighbour to be deleted could occur anywhere within a slab list. The traversal of the slab list halts once the adjacent edge to be deleted is found. The insertion operation requires adding new slabs once a slab list becomes full. In **Meerkat**, each thread in a warp processes slabs holding neighbors of the same vertex, resulting in better load balance and coalesced memory access.

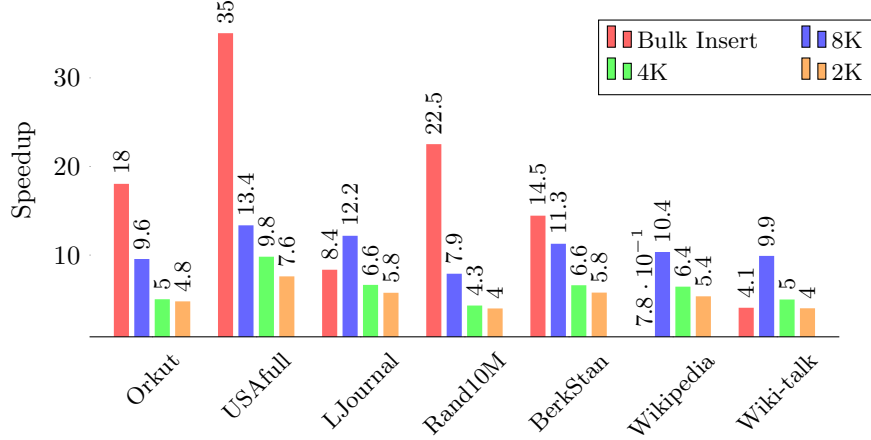


Figure 3: Insertion - Speedup Over HORNET

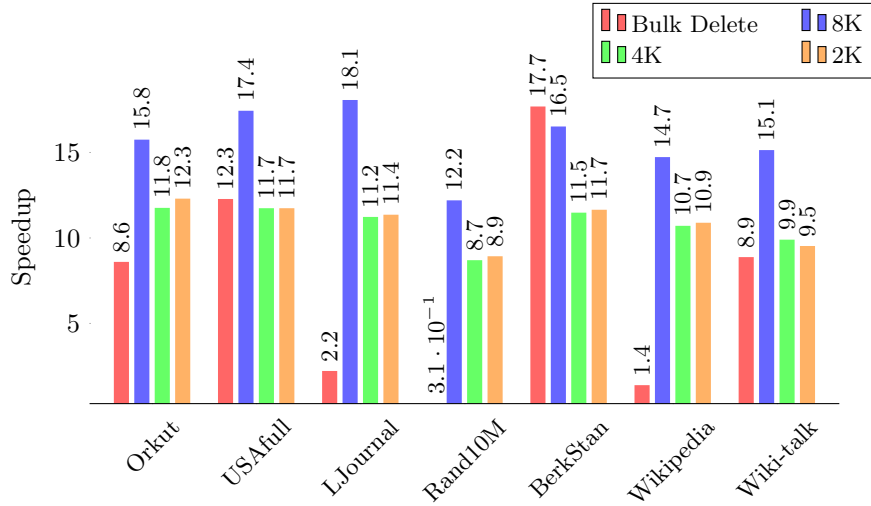


Figure 4: Deletion - Speedup Over HORNET

6.1 BFS and SSSP

The BFS and SSSP algorithms are programmed in **Meerkat** using two approaches. The VANILLA implementation uses 32-bit atomics and the TREE-BASED implementation uses 64-bit atomics. The VANILLA implementation only computes the shortest distances for reachable vertices from the source vertex. The TREE variant, however, also computes the dependency tree tracking how these distances have been computed. This dependency maintenance is necessary for the correct working of our incremental/decremental algorithms. The BFS and SSSP algorithms in **Meerkat** and HORNET follow an iterative level-based approach [18] using a pair of frontiers.

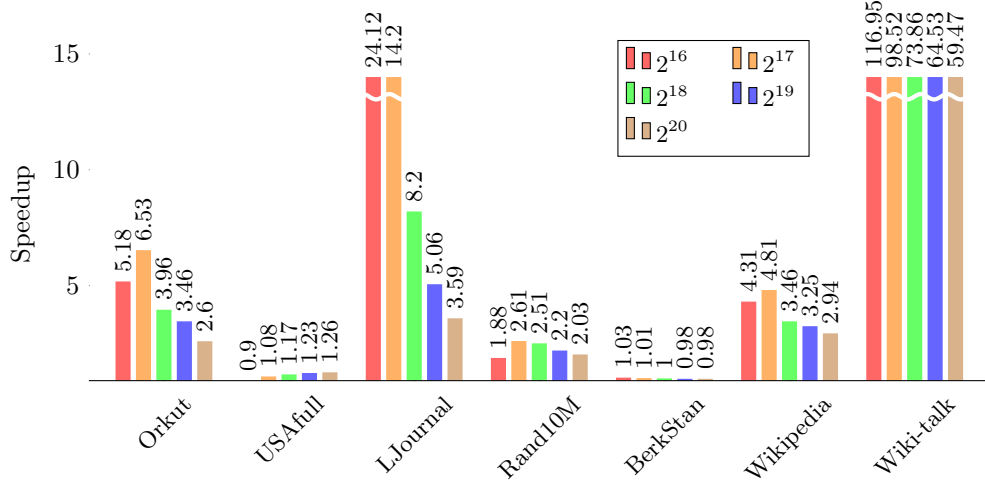


Figure 5: Query - Speedup Over HORNET

Populating the frontier for the next iteration $i + 1$ requires traversal over the outgoing edges of the destination vertices of the edges in the current iteration i . It is prudent to disable hashing for the BFS and SSSP benchmarks since it forces the maintenance of a single slab list for every vertex. This has a direct consequence in improving the slab occupancy, especially in graphs that have a high average out-degree (such as Orkut, Higgs, and Wikipedia), and in reducing the number of allocated slabs to store the initial graph. On disabling hashing, the average slab occupancy improves by 24% for Orkut, 14.35% for Higgs, 8% for Pokec and 5% for LJournal, with 6.26% improvement across all our benchmark graphs.

Disabling hashing for the BFS algorithm produces an average of 10.78% improvement (up to 28.1%) in performance. Similarly, an average of 9.1% improvement (up to 23.28%) in performance for TREE-BASED variant. Similarly, disabling hashing for the SSSP benchmark produces an average of 9.9% improvement (upto 35%) in performance. The TREE-BASED variant shows a similar average improvement of 11% (upto 28.95%).

6.1.1 Static BFS and SSSP comparison with *Hornet*

The Figure 6a shows the speedup of the static VANILLA and the TREE-BASED implementation of BFS in *Meerkat* over the static implementation in HORNET. The Figure 6b shows the speedup of the static VANILLA and TREE-BASED SSSP algorithm on *Meerkat* against that of HORNET. The TREE-BASED approach is necessary for setting up the initial data structures for incremental/decremental variants of our BFS/SSSP implementations on *Meerkat*. Our VANILLA BFS algorithm on *Meerkat* is on an average, $1.17\times$ (upto $1.82\times$) faster than that of HORNET. The VANILLA SSSP algorithm on *Meerkat* is on average, $1.32\times$ (upto $1.85\times$) faster than HORNET’s implementation. The reasons for the speedup in *Meerkat* are better-coalesced access memory access, and lack of migration of memory blocks.

The TREE-BASED BFS and SSSP algorithm on *Meerkat*, in contrast to the VANILLA implementations initialize tree-nodes (a 64-bit sized $\langle distance_{SRC}, parent \rangle$ pair) using 64-bit atomics. The TREE-BASED BFS has an average overhead of 17.2% over the VANILLA version, seen in the execution

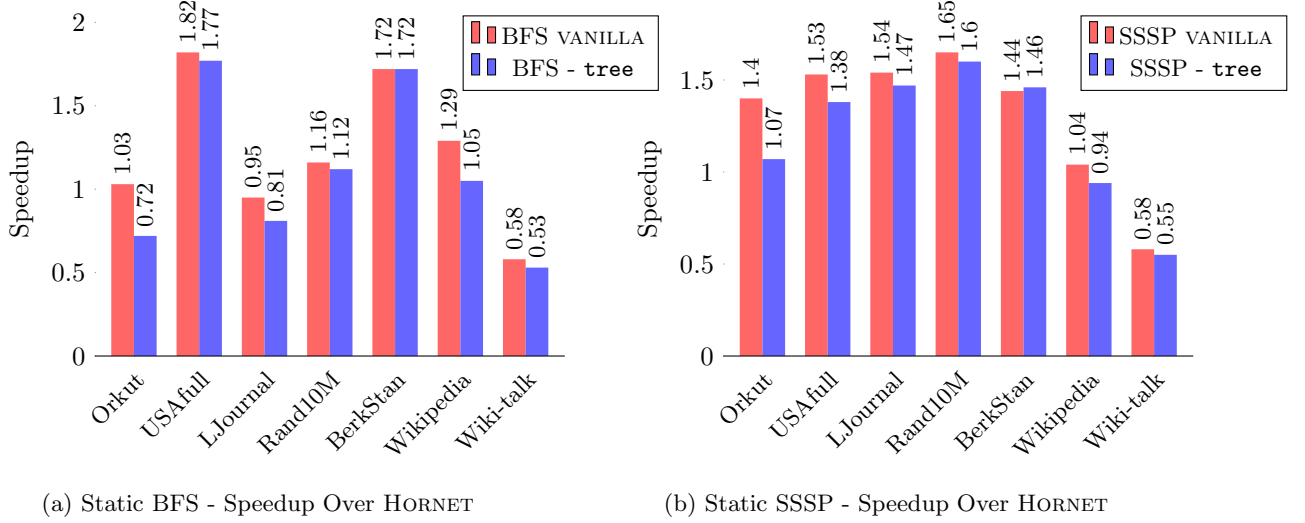


Figure 6: Static BFS / SSSP

time. A similar overhead of $\approx 14\%$ was seen in the case of TREE-BASED SSSP.

6.1.2 Dynamic BFS and SSSP in Meerkat

Figures 7a and 7b show the s_{10k}^{10} incremental and decremental speedups with respect to the static algorithms on **Meerkat**, for dynamic BFS and SSSP, respectively. The incremental BFS and SSSP are bound to be faster than the decremental algorithm. The incremental BFS and SSSP are performed by choosing the input batch of edges as the initial frontier and iterative application of the static algorithm to recompute the TREE. The decremental variant involves invalidation of affected vertices in the TREE, the propagation of invalidation up the TREE, computation of the initial frontier from unaffected vertices and re-computation of the TREE invariant by iterative application of the static algorithm.

With exception to USAfull and BerkStan, the execution times of repeated application of the static algorithm were on an average 7.3% and 5.6% lower than static running time on the original graph, for incremental BFS and incremental SSSP respectively. For USAfull and BerkStan, this difference was close to 80% and 71% respectively. In the case of incremental BFS on USAfull, we observed that there was an $11.53\times$ decrease in the average distance after the addition of first 10K, and nearly $2\times$ decrease from the first to the tenth batch. No significant increase in the number of reachable vertices was observed. In the case of BerkStan, while the average distance decreased from 11.7 to 8.58 for our sequence of ten incremental batches of 10K, we observed that number of reachable vertices increased from $\approx 460K$ to $\approx 591K$. Due to this graph topology, the speedup for incremental BFS/SSSP was much lower USAfull and BerkStan compared to other graphs. We have not seen any significant increase in the number of reachable vertices or a decrease in the average distance for other benchmark graphs.

In the case of decremental BFS and SSSP, the number of edges in the dependence tree that have been invalidated depends on the average in-degree of the vertex. We observed that for low average in-degree graphs, the likelihood of tree edges being invalidated was higher than those of

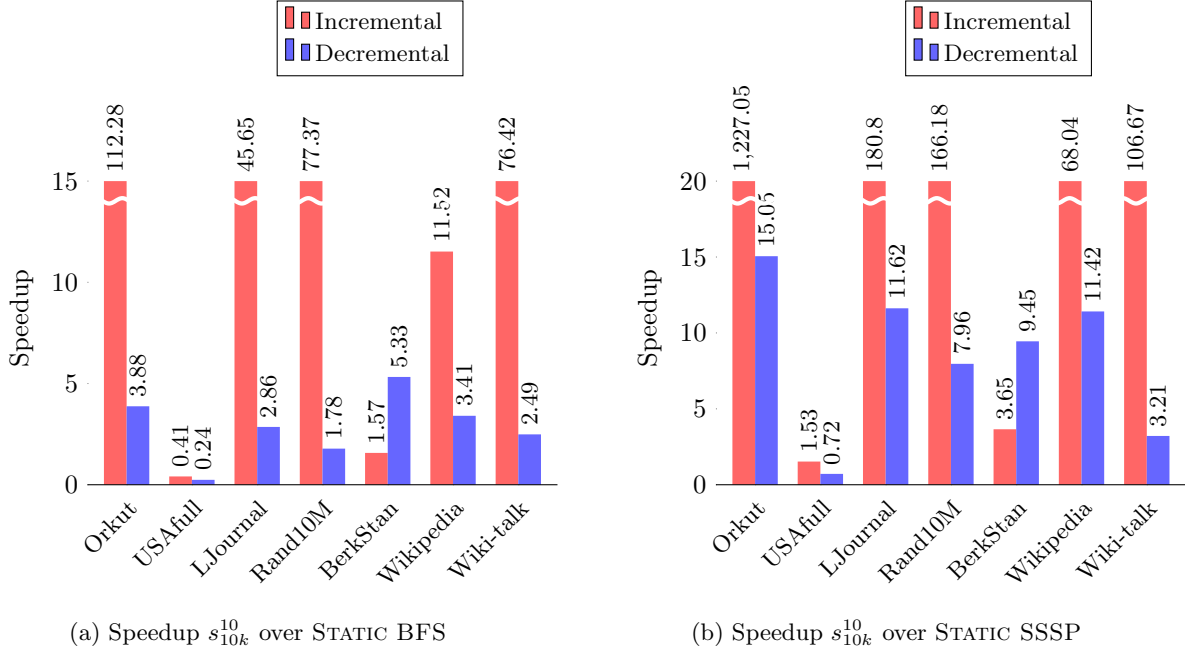


Figure 7: Dynamic BFS and SSSP on Meerkat

high in-degree graphs. For example, in the case of decremental BFS for a sequence of ten 10K batches, USAfull (with an average in-degree 2), an average of 38.97% of the decremental batch were tree edges, while it was 0.769% of the decremental batch for Orkut (with an average in-degree 72). It must be understood that the depth of the dependence tree is the BFS distance. Smaller tree depth (BFS distance) and large average degree favors only fewer vertices to be invalidated. In our observation of decremental BFS for 10K batches, we saw an average of 0.23K vertices for Orkut, 0.4K for Wikipedia, 1K for LJournal, 3.94K for BerkStan, 6K vertices for Rand10M, whose distances were invalidated, while it was an average of 9.54M vertices for USAfull, after each batch. This explains why USAfull performs poorly with our decremental BFS/SSSP algorithm. In the case of BerkStan, we have seen a decrease in the average distance for successive decremental batches, while other graphs have shown a marginally increasing trend in the average distance for successive decremental batches. The number of reachable vertices reduced by $\approx 2\%$ after ten batches; Rand10M and Orkut did not show any decrease, and the decrease was on an average 0.047% (upto 0.18% for USAfull) for other graphs.

6.2 PageRank

For our experimental evaluation of PageRank, we have set the damping factor to be 0.85, and the error margin to be 0.00001. The computation of PageRank (See Algorithm 5) involves the traversal of neighbours of each vertex, along their incoming edges. The same was the case with SSSP and BFS, but with traversal of neighbours, along their outgoing edges. Hashing was disabled in the Pagerank implementation as done in SSSP and BFS.

Disabling hashing improves the slab occupancy, especially in graphs with a higher average in-

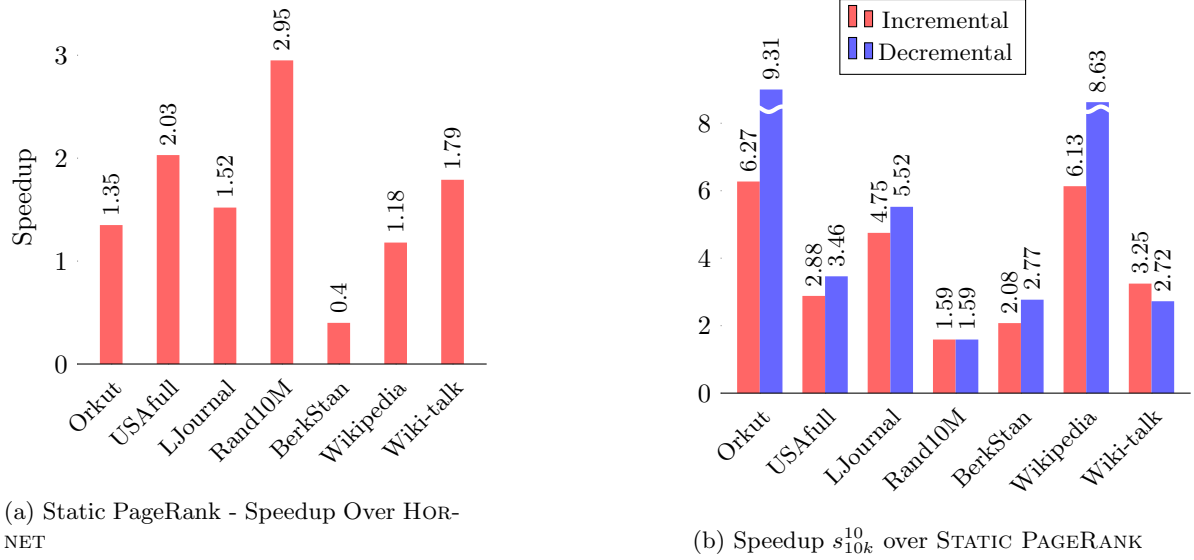


Figure 8: Static / Dynamic PageRank

degree. For low average in-degree graphs such as USAfull (2), Rand10M (8), and Wiki-talk (2), there is no performance improvement observed, as disabling hashing has virtually no effect: most vertices owing to their low indegree have single slab lists. However, for large average indegree graphs such as Orkut (76) and Wikipedia (27), disabling hashing produces a speedup of about 1.36-1.62 $\times$ improvement in the static PageRank running time.

6.2.1 Comparison of Static implementation with *Hornet*

The Figure 8a compares the performance of static PageRank on *Meerkat*, with that of HORNET. It is observed that in six out of seven graphs(that is, except Rand10M), *Meerkat* performs 1.18-2.49 $\times$ (with an average of 1.74 $\times$) faster than HORNET. The PAGERANK implementation on both *Meerkat* and HORNET are traversal based algorithms. Each iteration applies the computation of PAGERANK on all vertices. The number of iterations depends on when the convergence is achieved based on the convergence strategy. The performance improvement in *Meerkat* can be attributed to our efficient iterators performing coalesced accesses in retrieving adjacent vertices. While HORNET attempts to avoid warp-divergence, its traversal mechanism does not perform coalesced accesses.

6.2.2 Speedup of Dynamic Implementation in Meerkat

The incremental and decremental algorithms are identical: the same static-PAGERANK algorithm is applied on the entire graph after performing insertion/deletion of edges, respectively. Figure 9 and Figure 10 show the average running time (in $\times$) and the average number of iterations (in $\circ$) for small incremental/decremental batches ranging from 1K to 10K, respectively. We observe an increasing trend in the execution time as the batches increase in size. The trend in the execution time bears a striking resemblance with the number of iterations. The number of vertices, whose PAGERANK values are invalidated, increases with the batch size.

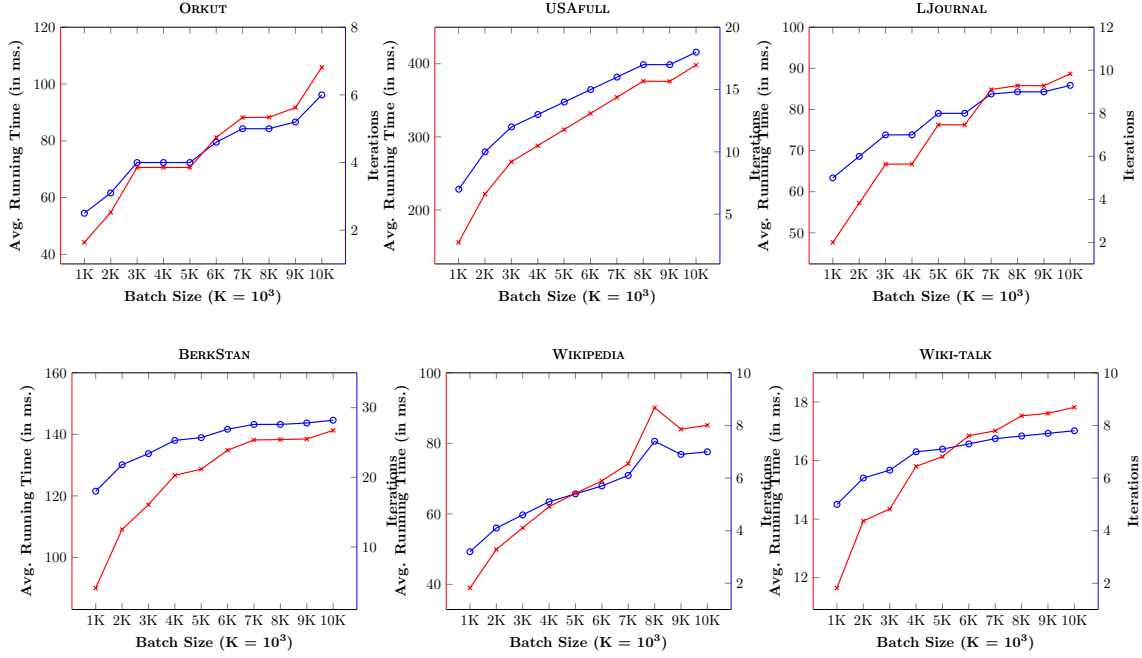


Figure 9: PAGERANK (INCREMENTAL - SMALL BATCHES)

Since all vertices participate in the computation of incremental/decremental PAGERANK, the running time must depend on the number of vertices. Further, the average execution time per iteration increases with the increase in the number of vertices. USAfull has the highest number of vertices and exhibits the highest running time among all graphs. Rand10M also has a high running time for similar reasons. Across, the incremental (decremental) batches, BerkStan has $1.8(1.57)\times$ higher number of iterations compared to USAfull, respectively. BerkStan has a smaller number of vertices but a higher diameter compared to other graphs. Owing to their small diameters, Orkut, LJJournal, Pokec, and Rand10M converge with fewer iterations, and show the slowest growth across 1K to 10K.

Figure 8b shows the s_{10k}^{10} incremental/decremental speedups with respect to the static algorithms on **Meerkat**. The speedup is directly proportional to the measure by which fewer iterations are required until convergence is achieved. For our batches of 10K batches of edges, we observed that Orkut achieved convergence with $\approx 20\%$ and $\approx 13\%$ of iterations required for the static variant. Whereas, the Rand10M converged in $\approx 64\%$ of iterations required for the static variant, for both incremental and decremental algorithms, registering the slowest speedup.

6.3 Triangle Counting

Figure 11a compares the performance of the static triangle counting algorithm on **Meerkat** against HORNET. It is observed in our experimental evaluation that HORNET performs on an average $31.12\times$ (upto $59.34\times$) faster than that of **Meerkat** on our benchmark graphs. The implementation of triangle counting on HORNET pre-processes the input batch, sorting the edges so that adjacent

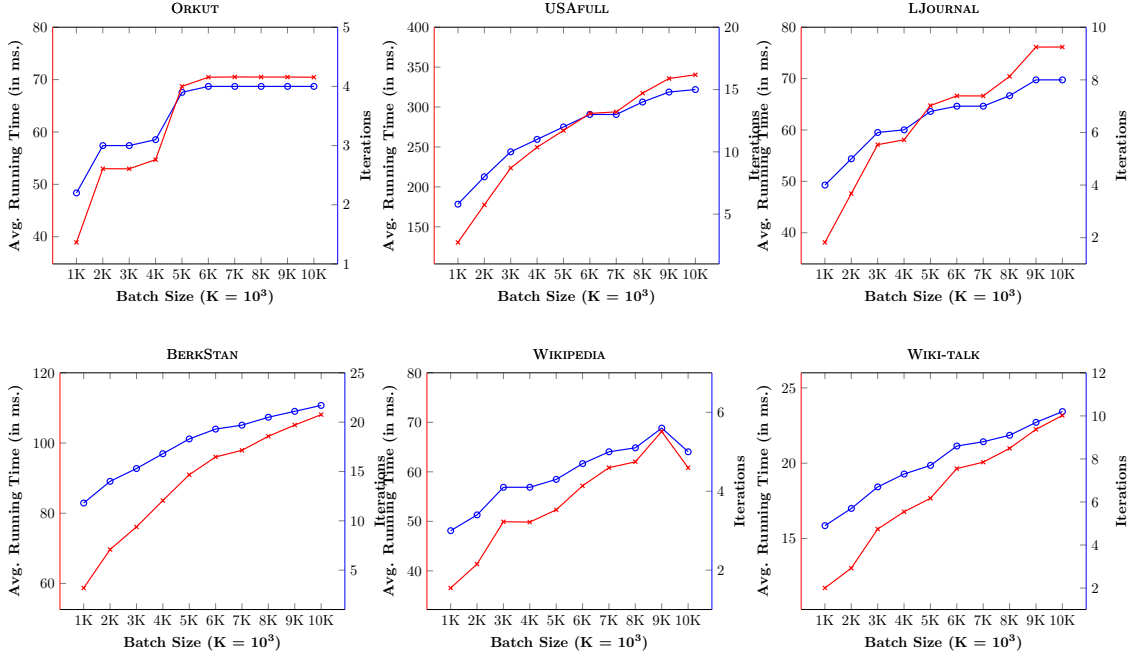


Figure 10: PAGERANK (DECREMENTAL - SMALL BATCHES)

neighbours of every vertex can be accessed in ascending order. This ordering of edges is beneficial for performing intersections of the adjacencies of the endpoints of an edge. In **Meerkat**, the neighbours of a vertex are stored in fixed-size slabs. While these neighbours are contiguous within a slab, the slabs themselves are not contiguous with each other. In **HORNET**, all the neighbours of a vertex are contiguously available within an edge block, whose size is the smallest power of two greater than the number of adjacent neighbours.

Firstly, **HORNET** divides the edges into multiple bins. In each bin, edges have the same estimated effort for performing an intersection of the adjacencies of their endpoints. Each bin of edges is processed in a separate kernel invocation so that proper load balancing is achieved among the kernel threads. **HORNET** uses two methods for performing the intersection of the adjacencies of the endpoints of the edges, which are dependent on their storage in ascending order. In the first method, **HORNET** chooses the endpoint having the smallest adjacency list of the two. It traverses through the vertices in this adjacency list and counts the membership in the adjacency list of the other vertex, by using binary search. The second method is based on a simultaneous traversal of both adjacency lists. A pointer is associated with each adjacency list; the pointer is advanced if it points to an element smaller than that referred to by the other pointer. The intersection count is incremented if both pointers refer to the same element. The drawback of forcing an ascending order among the adjacencies for each vertex is that it makes the graph rigid for dynamic mutation.

In contrast, the presence of hashing in **Meerkat** distributes the adjacent vertices among multiple slabs, and the ordering among elements cannot be enforced. **Meerkat** cannot use efficient methods which **HORNET** can use due to this lack of ordering of edges. In **Meerkat**, for each edge $u \rightarrow v$ (assuming u has a smaller adjacency than v without loss of generality), and the slabs of u are first

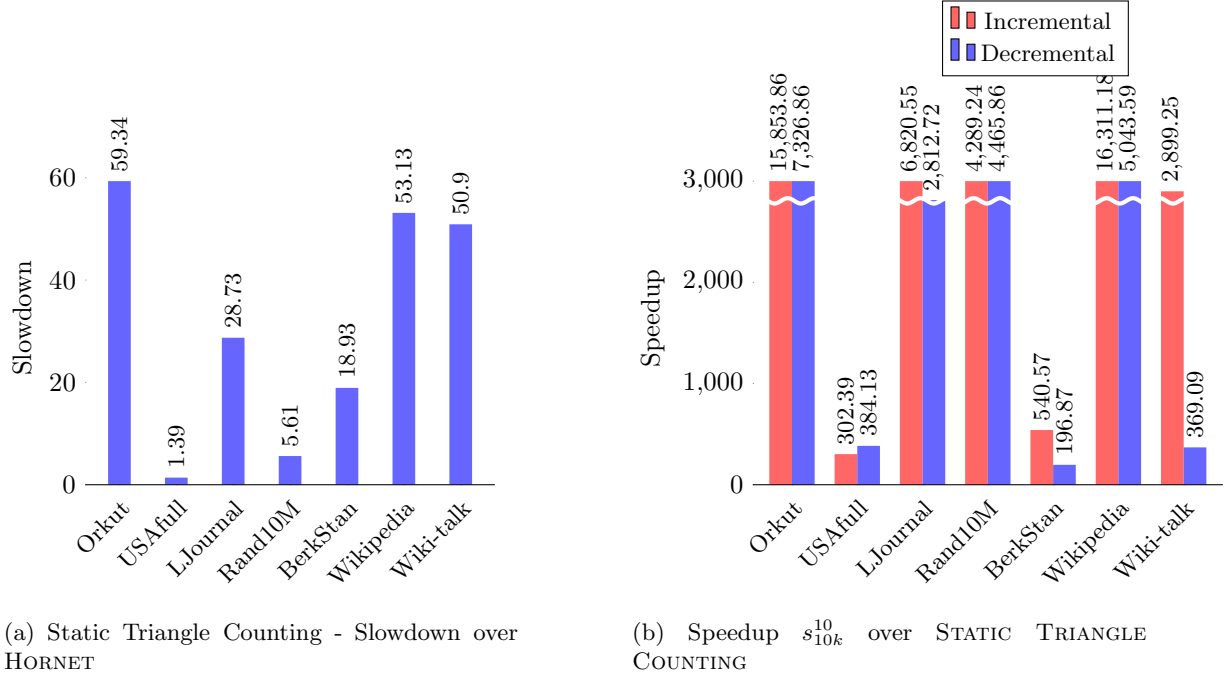


Figure 11: Static / Dynamic Triangle Counting on **Meerkat**

traversed with the help of our iterators. For each vertex w adjacent to u retrieved by our iterators, the `SEARCHEDGE()` device method checks for the existence of w in the adjacencies of v . This is achieved by identifying the slab list using hashing with vertex w to identify the slab list that could potentially hold w , and by traversing the slab list to discover the vertex w .

The BFS/SSSP and PAGERANK benchmarks which have a traversal component, benefit from disabling the hashing mechanism, since it improves the slab occupancy and reduces the number of slabs to traverse per vertex. The traversal operations in such benchmarks must visit all the slabs of vertex whose adjacencies have to be inspected. However, enabling hashing for the TRIANGLE COUNTING benchmark improves the performance by $15.44\times$ on our benchmark graphs. Enabling hashing distributes the slabs among multiple slab lists; only the slab list that could potentially accommodate the search vertex can be inspected. This reduces the number of slabs to inspect while performing `SEARCHEDGE()` operation during the intersection operation.

Figure 11b shows the s_{10k}^{10} speedup of our incremental/decremental algorithms over the static algorithms on **Meerkat**. Across the benchmarks, superlative speedups are observed since, for each batch, the static algorithm counts the number of triangles by performing an intersection for the adjacencies of both end-points for every graph edge, while the dynamic algorithm performs intersection only for the end-points of the edges in the batch. The speedup observed is very large if the batch size is very small compared to the number of edges in the graph. Hence, graphs such as Orkut, LJournal, Rand10M, and Wikipedia enjoy very high speedups compared to the repeated application of the static algorithm.

6.4 Weakly Connected Component (WCC)

We evaluate the performance of the static WCC algorithm on **Meerkat** against HORNET, followed by, the performance of incremental WCC in **Meerkat** for various optimizations. The decremental WCC on GPU, at the time of writing, is an unsolved problem.

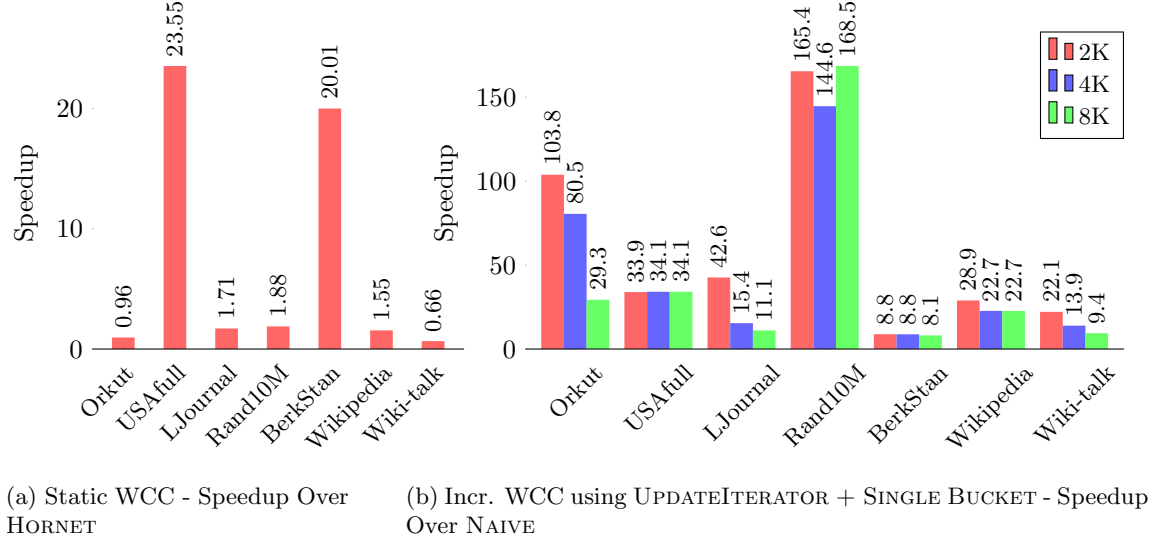


Figure 12: Weakly Connected Components

6.4.1 Static WCC in Meerkat vs. Hornet

Figure 12a compares the performance of static WCC on **Meerkat** against HORNET. HORNET uses a modified-BFS like algorithm discovering connected components, using a two-level queue. With a two-level queue, the `insert()` and `deque()` operations are performed on two separate queues. In the first step, the discovery of the largest connected component is attempted. A BFS is performed from the source vertex with the help of a two-level queue, and all the reachable vertices are marked with the same color. In the second step, all the unvisited vertices are incrementally assigned a unique color. In the third step, the edges from these unreachable vertices are queued, and both end-points are iteratively assigned the smallest of their colors. This iterative process continues until all the endpoints of such edges have the same color. In **Meerkat**, the static WCC implementation uses the union-find approach for discovering weakly-connected components. It performs a single traversal through all the adjacencies of the graph: it uses the UNION-ASYNC strategy for the UNION operation for the adjacent edges discovered, and full path compression for determining the representative elements for the vertices in FIND operation.

We observe that while **Meerkat** performs $6.08\times$ on an average across all our input graphs, the speedup against HORNET is lower if there are vertices with very large out-degree. This is observed in graphs such as Orkut, LJournal, and Wikipedia. This is because a large-out degree vertex will cause many vertices to be enqueued into the BFS frontier queue, thereby improving parallelism.

However, for networks such as USAfull, BerkStan, the diameter of such graphs is much higher, the BFS-approach in HORNET performs significantly worse compared to **Meerkat**.

6.4.2 Incremental WCC in Meerkat

Incremental WCC is evaluated with different schemes.

- *Naive*: traverses through all the slab lists as it is ignorant about the location of the new updates. Hence, it is expected to be the least performant implementation. This algorithm is identical to the naive static WCC algorithm on SlabGraph.
- *SlabIterator*: maintains a boolean flag for every vertex. This flag is set to true for a vertex if there are new adjacent vertices in the input batch. All the neighboring adjacencies of such a vertex are traversed. This bears the potential to significantly reduce the need to visit a large number of source vertices, especially when the batch size is small compared to the current graph, and when a source vertex is shared by a significant majority of batch update edges.
- *UpdateIterator*: apart from maintaining a boolean flag to identify source vertices with adjacencies, this implementation maintains an update flag for every slab list and the allocator address of the first updated slab. Unlike the *SlabIterator*, *UpdateIterators* allow visitation of only those slabs storing the new updates. flag along with the *UpdateIterator*.
- *UpdateIterator + Single Bucket*: Evaluates the *UpdateIterator* approach in the absence of hashing. Intuitively, this should ensure that a warp operating on an *UpdateIterator* would see more updates in single memory access while traversing a slab list. This method produced the highest speedup with respect to the *Naive* variant .

Method	Dataset	2K	4K	8K
SlabIterator	LJournal	40.5×	12.3×	10.9×
	Rand10M	149.8×	170.4×	146.9×
	BerkStan	9.55×	9.98×	9.03×
	Wiki-talk	18.3×	8.62×	7.37×
	Wikipedia	23.65×	18.19×	18.07×
	Orkut	82.3×	62.9×	20.5×
	USAfull	33.9×	34.3×	34.0×
UpdateIterator	LJournal	34.6×	8.36×	7.58×
	Rand10M	149.0×	146.8×	144.7×
	BerkStan	7.71×	7.82×	7.34×
	Wiki-talk	15.8×	7.27×	6.37×
	Wikipedia	19.56×	15.34×	15.08×
	Orkut	71.0×	52.9×	17.0×
	USAfull	33.9×	34.0×	33.6×

Table 6: Incremental WCC: Speedup over the naïve scheme

Figure 12b compares the performance of *UpdateIterator + Single Bucket* against the *naïve* scheme, while a similar comparison of the *SlabIterator* and *UpdateIterator* schemes is shown in Table 6. It must be remembered that the naïve scheme cannot distinguish between the updated slabs, and those slabs holding adjacent vertices previously inserted. Hence, its running time is proportional to the number of edges present in the graph representation. Our optimized processing iterates over only the updated slabs (*UpdateIterator*, *UpdateIterator + Single Bucket*)/slab-lists

(*SlabIterator*); therefore, the running time is proportional to the number of slabs/slab lists holding new updates, respectively. Hence, the speedup decreases with increasing batch size.

The use of a *update* flag for vertex contributes significantly to eliminating the traversal of the adjacencies of the source vertices which do not have updated slab lists. Almost identical performance is observed between the *UpdateIterator* and the *UpdateIterator + Single Bucket* schemes for low-degree graphs such as USAfull. If the average degree of a vertex is less than a slab’s capacity, only one slab list (one head slab) would be allocated per vertex, and most updates would thus fit the head slab comfortably. The extra overhead visible in the fourth major row would be attributed to checking the slab list’s *update* flag. In high-degree graphs such as social networks (namely, Orkut, Wikipedia, and Wiki-talk), the *UpdateIterator* over a single bucket overcomes previously inserted vertices, with the updates restricted contiguously to a single slab list, resulting in a marginal increase in performance. However, the use of *UpdateIterator* with the vertex flag and the allowance of multiple slab lists seems to have lower performance than the use of a single slab list for high-degree graphs that follow the *power-law* distribution: Orkut, Wikipedia, and Wiki-talk. In the presence of multiple slab lists, the *UpdateIterator* must sequentially probe if the *update* flag for a slab list is set for every slab list of the source vertex. This overhead is overcome in the use of a single slab list by subsuming the function of the update flag of a slab list within that for the source vertex.

7 Conclusion and Future Work

We presented a new framework **Meerkat** for dynamic graph algorithms on GPUs. It builds upon and significantly enhances a hash-based SlabHash data structure. **Meerkat** offers a memory-efficient alternative, proposes new iterators, and optimizes their processing to improve both the execution time as well as the memory requirement. These enhancements allow dynamic graph algorithms, containing both incremental and decremental updates, to be implemented efficiently on GPUs. We illustrated the effectiveness of the framework using fundamental graph algorithms: BFS, SSSP, PR, TC, and WCC. As part of future work, we would like to implement more complex graph algorithms using our framework, and also check for the feasibility of approximations to reduce the memory requirement of **Meerkat** further.

References

- [1] 9th dimacs implementation challenge - shortest paths. <http://www.diag.uniroma1.it/challenge9/download.shtml>, May 2013.
- [2] ACAR, U. A., ANDERSON, D., BLELLOCH, G. E., AND DHULIPALA, L. Parallel batch-dynamic graph connectivity. In *The 31st ACM Symposium on Parallelism in Algorithms and Architectures* (New York, NY, USA, 2019), SPAA ’19, Association for Computing Machinery, pp. 381–392.
- [3] AFARIN, M., GAO, C., RAHMAN, S., ABU-GHAZALEH, N., AND GUPTA, R. Commongraph: Graph analytics on evolving data. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (New York, NY, USA, 2023), ASPLOS 2023, Association for Computing Machinery, pp. 133–145.

- [4] ARASU, A., NOVAK, J., TOMKINS, A., AND TOMLIN, J. A. Pagerank computation and the structure of the web: Experiments and algorithms.
- [5] ASHKIANI, S., FARACH-COLTON, M., AND OWENS, J. D. A dynamic hash table for the gpu. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2018), pp. 419–429.
- [6] ASHKIANI, S., FARACH-COLTON, M., AND OWENS, J. D. A dynamic hash table for the gpu. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2018), pp. 419–429.
- [7] AWAD, M. A., ASHKIANI, S., PORUMBESCU, S. D., AND OWENS, J. D. Dynamic graphs on the gpu. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2020), pp. 739–748.
- [8] BACKSTROM, L., HUTTENLOCHER, D., KLEINBERG, J., AND LAN, X. Group formation in large social networks: Membership, growth, and evolution. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, NY, USA, 2006), KDD '06, Association for Computing Machinery, pp. 44–54.
- [9] BENDER, M. A., AND HU, H. An adaptive packed-memory array. In *Proceedings of the Twenty-Fifth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems* (New York, NY, USA, 2006), PODS '06, Association for Computing Machinery, pp. 20–29.
- [10] BUSATO, F., GREEN, O., BOMBIERI, N., AND BADER, D. A. Hornet: An efficient data structure for dynamic sparse graphs and matrices on gpus. In *2018 IEEE High Performance extreme Computing Conference (HPEC)* (2018), pp. 1–7.
- [11] CHEN, D., GUI, C., ZHANG, Y., JIN, H., ZHENG, L., HUANG, Y., AND LIAO, X. Graphfly: Efficient asynchronous streaming graphs processing via dependency-flow. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis* (2022), SC '22, IEEE Press.
- [12] DE LEO, D., AND BONCZ, P. Teseo and the analysis of structural dynamic graphs. *Proc. VLDB Endow.* 14, 6 (feb 2021), 1053–1066.
- [13] DHULIPALA, L., BLELLOCH, G. E., AND SHUN, J. Low-latency graph streaming using compressed purely-functional trees. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (New York, NY, USA, 2019), PLDI 2019, Association for Computing Machinery, p. 918–934.
- [14] DHULIPALA, L., HONG, C., AND SHUN, J. Connectit: A framework for static and incremental parallel graph connectivity algorithms. *Proc. VLDB Endow.* 14, 4 (dec 2020), 653–667.
- [15] EDIGER, D., MCCOLL, R., RIEDY, J., AND BADER, D. A. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing* (2012), pp. 1–5.
- [16] FAGERBERG, R. *Cache-Oblivious B-Tree*. Springer US, Boston, MA, 2008, pp. 121–123.

- [17] FUCHS, P., MARGAN, D., AND GICEVA, J. Sortledton: A universal, transactional graph data structure. *Proc. VLDB Endow.* 15, 6 (feb 2022), 1173–1186.
- [18] GOGOI, B., CHERAMANGALATH, U., AND NASRE, R. Custom code generation for a graph dsl. In *Proceedings of the 13th Annual Workshop on General Purpose Processing Using Graphics Processing Unit* (New York, NY, USA, 2020), GPGPU '20, Association for Computing Machinery, p. 51–60.
- [19] GREEN, O., AND BADER, D. A. custinger: Supporting dynamic graph algorithms for gpus. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)* (2016), pp. 1–6.
- [20] HONG, C., DHULIPALA, L., AND SHUN, J. Exploring the design space of static and incremental graph connectivity algorithms on gpu. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (New York, NY, USA, 2020), PACT '20, Association for Computing Machinery, pp. 55–69.
- [21] JAIGANESH, J., AND BURTSCHER, M. A high-performance connected components implementation for gpu. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2018), HPDC '18, Association for Computing Machinery, pp. 92–104.
- [22] JAIYEoba, W., AND SKADRON, K. Graphtinker: A high performance data structure for dynamic graph processing. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2019), pp. 1030–1041.
- [23] KUMAR, P., AND HUANG, H. H. Graphone: A data store for real-time analytics on evolving graphs. *ACM Trans. Storage* 15, 4 (jan 2020).
- [24] LACKI, J. Improved deterministic algorithms for decremental reachability and strongly connected components. *ACM Trans. Algorithms* 9, 3 (jun 2013).
- [25] LESKOVEC, J., HUTTENLOCHER, D., AND KLEINBERG, J. Signed networks in social media. CHI '10, Association for Computing Machinery, pp. 1361–1370.
- [26] LESKOVEC, J., LANG, K. J., DASGUPTA, A., AND MAHONEY, M. W. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters, 2008.
- [27] MAKAR, D., BADER, D. A., AND GREEN, O. Exact and parallel triangle counting in dynamic graphs. In *2017 IEEE 24th International Conference on High Performance Computing (HiPC)* (2017), pp. 2–12.
- [28] MALHOTRA, G., CHAPPIDI, H., AND NASRE, R. Fast dynamic graph algorithms. In *Languages and Compilers for Parallel Computing - 30th International Workshop, LCPC 2017, College Station, TX, USA, October 11-13, 2017, Revised Selected Papers* (2017), L. Rauchwerger, Ed., vol. 11403 of *Lecture Notes in Computer Science*, Springer, pp. 262–277.
- [29] NVIDIA. Nvidia warp primitives, May 2023.
- [30] PANDEY, P., WHEATMAN, B., XU, H., AND BULUC, A. Terrace: A hierarchical graph container for skewed dynamic graphs. In *Proceedings of the 2021 International Conference on Management of Data* (New York, NY, USA, 2021), SIGMOD '21, Association for Computing Machinery, p. 1372–1385.

- [31] RAMALINGAM, G., AND REPS, T. On the computational complexity of dynamic graph problems. *Theor. Comput. Sci.* 158, 1-2 (may 1996), 233–277.
- [32] SHA, M., LI, Y., HE, B., AND TAN, K.-L. Accelerating dynamic graph analytics on gpus. *Proc. VLDB Endow.* 11, 1 (sep 2017), 107–120.
- [33] SOMAN, J., KISHORE, K., AND NARAYANAN, P. J. A fast gpu algorithm for graph connectivity. In *2010 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum (IPDPSW)* (2010), pp. 1–8.
- [34] SUTTON, M., BEN-NUN, T., AND BARAK, A. Optimizing parallel graph connectivity computation via subgraph sampling. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2018), pp. 12–21.
- [35] VORA, K., GUPTA, R., AND XU, G. Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems* (New York, NY, USA, 2017), ASPLOS '17, Association for Computing Machinery, pp. 237–251.
- [36] WANG, Q., ZHENG, L., HUANG, Y., YAO, P., GUI, C., LIAO, X., JIN, H., JIANG, W., AND MAO, F. Grasu: A fast graph update library for fpga-based dynamic graph processing. In *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (New York, NY, USA, 2021), FPGA '21, Association for Computing Machinery, pp. 149–159.
- [37] WINTER, M., MLAKAR, D., ZAYER, R., SEIDEL, H.-P., AND STEINBERGER, M. Faimgraph: High performance management of fully-dynamic graphs under tight memory constraints on the gpu. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (2018), SC '18, IEEE Press.
- [38] WINTER, M., ZAYER, R., AND STEINBERGER, M. Autonomous, independent management of dynamic graphs on gpus. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)* (2017), pp. 1–7.
- [39] YANG, J., AND LESKOVEC, J. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics* (New York, NY, USA, 2012), MDS '12, Association for Computing Machinery.
- [40] ZHANG, F., ZOU, L., AND YU, Y. Lpma - an efficient data structure for dynamic graph on gpus. In *Web Information Systems Engineering – WISE 2021* (Cham, 2021), W. Zhang, L. Zou, Z. Maamar, and L. Chen, Eds., Springer International Publishing, pp. 469–484.
- [41] ZHANG, Y., LIANG, Y., ZHAO, J., MAO, F., GU, L., LIAO, X., JIN, H., LIU, H., GUO, S., ZENG, Y., HU, H., LI, C., ZHANG, J., AND WANG, B. Egraph: Efficient concurrent gpu-based dynamic graph processing. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2023), 5823–5836.
- [42] ZHU, X., FENG, G., SERAFINI, M., MA, X., YU, J., XIE, L., ABOULNAGA, A., AND CHEN, W. Livegraph: A transactional graph storage system with purely sequential adjacency list scans. *Proc. VLDB Endow.* 13, 7 (mar 2020), 1020–1034.

A Dynamic Algorithms using Meerkat

A.1 Dynamic Triangle Counting

Algorithm 7: Triangle Counting - Incremental

```

1 function TC_Incremental (Graph PostInsertionGraph, Graph UpdateGraph, Edge edges[edge_n]) {
2   unsigned int  $S_1^i = 0, S_2^i = 0, S_3^i = 0$ 
3   Count(PostInsertionGraph, PostInsertionGraph, edges,  $S_1^i$ )
4   Count(PostInsertionGraph, UpdateGraph, edges,  $S_2^i$ )
5   Count(UpdateGraph, UpdateGraph, edges,  $S_3^i$ )
6   return  $0.5 \times (S_1^i - S_2^i + S_3^i / 3)$ 
7 }

```

Algorithm 8: Triangle Counting - Decremental

```

1 function TC_Decremental (Graph PostDeletionGraph, Graph UpdateGraph, Edge edges[edge_n]) {
2   unsigned int  $S_1^d = 0, S_2^d = 0, S_3^d = 0$ 
3   Count(PostDeletionGraph, PostDeletionGraph, edges,  $S_1^d$ )
4   Count(PostDeletionGraph, UpdateGraph, edges,  $S_2^d$ )
5   Count(UpdateGraph, UpdateGraph, edges,  $S_3^d$ )
6   return  $0.5 \times (S_1^d + S_2^d + S_3^d / 3)$ 
7 }

```

Algorithms 7 and 8 show the computation of number of triangles added/deleted after an incremental/decremental batch of edges, respectively.

The *Count*($G_1, G_2, edges, count$) GPU kernel, shown in Algorithm 9 computes the total number of intersections found in the adjacencies of the end-points of all edges (u, v) , for $u \in V(G_1)$, and $v \in V(G_2)$.

A.2 Dynamic Single Source Shortest Path and Breadth First Search

Algorithm 6 invokes the SSSP computation kernel (described in Algorithm 6) for static SSSP computation (in line 6), and in the common epilogue for incremental and decremental SSSP (line 24). Similarly, Algorithms 11 and 12 are invoked by the prologue of the decremental SSSP algorithm (at Algorithm 6, lines 18 and 19, respectively) for invalidation of vertices, and its propagation, in the SSSP value computation dependence tree.

A.3 Dynamic PageRank

Algorithm 14 (invoked in Algorithm 5, line 9) describes the GPU kernel for computing the PageRank values for all vertices of our graph.

Algorithm 13 (invoked in Algorithm 5, line 11) is used for computing the probability of teleporting from vertices with zero out-degree.

Algorithm 9: Triangle Counting - Count Kernel

```
1 function Count(Graph  $G_1$ , Graph  $G_2$ , Edge edges[edge_n], int *TotalCount) {
2     /* Incremental/decremental batch of edges stored in array edges[] */
3     VertexDictionary *vert_adj[] = G2.get_vertex_adjacencies()
4     if ((thread_id() - lane_id() < edge_n)) then
5         bool to_count = (thread_id() % edge_n)
6         unsigned int count = 0
7         unsigned int32 work_queue = 0
8         Vertex u = INVALID_VERTEX
9         Vertex v = INVALID_VERTEX
10        if (to_count == true) then
11            /* Assign (u, v) to thread. */
12            u = edges.src[thread_id()]
13            v = edges.dst[thread_id()]
14        end if
15        unsigned int32 dequeue_lane = 0
16        /* Follows IterationScheme1 */
17        while ((dequeue_lane = warpdequeue(&to_compute))  $\neq$  -1) do
18            Vertex current_u = warpbroadcast(&u, current_lane)
19            Vertex current_v = warpbroadcast(&v, current_lane)
20            /* Construct iterators for traversal of adjacent slabs for vertex v in  $G_2$  */
21            SlabIterator iter = vert_adj[current_v].begin()
22            SlabIterator last = vert_adj[current_v].end()
23            while (iter  $\neq$  last) do
24                /* Loop performs intersection of the adjacencies of u in  $G_1$  and v in  $G_2$  */
25                Vertex adj_v = *(iter.get_pointer(lane_id()))
26                bool edge_present =  $G_1$ .SearchEdge(is_valid_vertex(adj_v), current_u, adj_v)
27                if (edge_present) then
28                    /* Update thread-local count if intersection found */
29                    count += 1
30                end if
31                ++ iter
32            end while
33        end while
34        /* Warp-level reduction of intersection count */
35        warpreduxsum(&count)
36        if (lane_id() == 0 and count  $\neq$  0) then
37            /* Update global intersection count. Each warp performs atmost one atomicAdd() */
38            atomicAdd(TotalCount, count)
39        end if
40    end if
41 }

```

Algorithm 10: Static SSSP - Computation Kernel

```
1 function SSSP_Kernel(Graph G, tree_node D[vertex_n], Frontier<Edge> Fcurrent,  
  Frontier<Edge> Fnext ) {  
2   VertexDictionary *vert_adj[] = G.get_vertex_adjacencies()  
3   if ((thread_id() - lane_id()) < Fcurrent.size) then  
4     Edge e = INVALID_EDGE  
5     bool to_consider = false  
6     if (thread_id() < Fcurrent.size) then  
7       e = Fcurrent[thread_id()]  
8       /* Compute tree node for vertex e.dst */  
9       tree_node ddst = <D[e.src].distance + e.wgt, e.src>  
10      /* Atomically update tree node for e.dst */  
11      to_consider = atomicMin(&D[e.dst], ddst) > ddst  
12    end if  
13    unsigned int32 dequeue_lane = 0  
14    /* Loop enqueues outgoing neighbours of e.dst, if its tree node was updated */  
15    while ((dequeue_lane = warpdequeue(&to_consider)) ≠ -1) do  
16      Vertex current_v = warpbroadcast(e.dst, dequeue_lane)  
17      SlabIterator iter = vert_adj[current_v].begin()  
18      SlabIterator last = vert_adj[current_v].end()  
19      while (iter ≠ last) do  
20        /* Iterate over neighbours */  
21        Pair p<dst, weight> = *(iter.get_pointer(lane_id()))  
22        Edge enext = INVALID_EDGE  
23        if (is_valid_pair(p)) then  
24          enext = {current_v, p.dst, p.weight}  
25        end if  
26        /* Enqueue outgoing edges of e.dst into next frontier Fnext */  
27        warpenqueuefrontier(Fnext, enext, enext ≠ INVALID_EDGE)  
28        ++ iter  
29      end while  
30    end while  
31  end if  
32 }
```

Algorithm 11: Decremental SSSP - Invalidate Distance Kernel

```
1 function Invalidate (Edges batch_edges[N], tree_node D[vertex_n]) {  
2   uint t = thread_id()  
3   if (t < N) then  
4     Vertex dst = batch_edges[t].dst  
5     /* Invalidate tree node for vertex dst if (parentdst, dst) is a batch edge for  
6       deletion */  
7     if (D[dst].parent == batch_edges[t].src) then  
8       D[dst] = <INF, INVALID>  
9     end if  
10  end if  
11 }
```

Algorithm 12: Decremental SSSP - Propagate Invalidation Kernel

```
1 function PropagateInvalidation (tree_node D[vertex_n], Vertex src) {
2   uint i = threads_id()
3   /* Grid-stride loop: each thread checks if the vertex i is reachable to source
4      src in the dependence tree */
5   while (i < vertex_n) do
6     if (D[i] ≠ ⟨INF, INVALID⟩) then
7       Vertex ancestor = D[i].parent
8       while (ancestor ≠ src) do
9         /* Traverses to the source vertex src: loops until src or invalidated
10            vertex is found in path */
11         tree_node da = D[ancestor]
12         if (da == ⟨INF, INVALID⟩) then
13           /* Invalidate vertex i if ancestor a is invalidated */
14           D[i] = ⟨INF, INVALID⟩
15           break
16         end if
17         ancestor = D[da].parent
18       end while
19     end if
20     i += threads_n()
21   end while
22 }
```

Algorithm 13: PageRank Algorithm - Find Teleport Value Kernel

```
1 function FindTeleportProb (uint OutDegrees[vertex_n], float OldPageRanks[vertex_n], float
2   *TeleportProb) {
3   uint stride = get_num_threads()
4   uint I = thread_id()
5   float local = 0.0
6   /* Uses grid-stride loop for accumulating contribution to teleportation
7      probability */
8   while (I < vertex_n) do
9     uint out_degree = OutDegrees[I]
10    if (out_degree == 0) then
11      local += OldPageRanks[I] / vertex_n
12    end if
13  end while
14  /* Warp-wide reduction of contributions to teleportation probability */
15  warpreduxsum(&local)
16  if (lane_id() == 0 and local ≠ 0.0f) then
17    /* Update global TeleportProb. Each warp performs atmost one atomicAdd() */
18    atomicAdd(TeleportProb, local)
19  end if
20 }
```

Algorithm 14: Page Rank - Compute Kernel

```
1 function Compute (Graph G, float VertexContribution[vertex_n], float DampingFactor, float  
   NewPageRanks[vertex_n]) {  
2   Vertex_Dictionary *vert_adj[] = G.get_vertex_adjacencies()  
3   if ((thread_id() - lane_id() < edge_n)) then  
4     bool to_compute = (thread_id() % VertexN)  
5     unsigned int pr_value = 0  
6     /* Follows IterationScheme1 */  
7     unsigned int32 dequeue_lane = 0  
8     while ((dequeue_lane = warpdequeue(&to_compute)) ≠ -1) do  
9       Vertex current_v = thread_id() - lane_id() + dequeue_lane  
10      /* Construct iterators for traversal */  
11      SlabIterator iter = vert_adj[current_v].begin()  
12      SlabIterator last = vert_adj[current_v].end()  
13      float local_prsum = 0.0f  
14      while (iter ≠ last) do  
15        /* Traversal of slabs using SlabIterators */  
16        Vertex u = *(iter.get_pointer(lane_id()))  
17        if (is_valid_vertex(u)) then  
18          /* Accumulation of thread-local vertex contributions to  
19          PageRank[current_v] */  
20          local_prsum += VertexContribution[u]  
21        end if  
22        ++ iter  
23      end while  
24      /* Warp-wide reduction of thread-local vertex contributions */  
25      warpreduxsum(&local_prsum)  
26      if (thread_id() == current_v) then  
27        /* Update thread-local PageRank value after warp-wide reduction */  
28        pr_value = local_prsum  
29      end if  
30    end while  
31    /* Update PageRanks in coalesced fashion */  
32    if (thread_id() < vertex_n) then  
33      NewPageRanks[thread_id()] = pr_value  
34    end if  
35  end if  
36 }  
}
```
